



Detecting AI-generated essays using fine-tuned XLNet-CNN hybrid techniques: a study of the academic integrity challenge

Manish Prajapati¹ · Santos Kumar Baliarsingh¹ · Prabhu Prasad Dev¹

Received: 22 January 2025 / Accepted: 11 November 2025 / Published online: 2 February 2026
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

The rapid advancement of large language models (LLMs) such as GPT-3 and GPT-4 has raised serious concerns for academic integrity, as AI-generated essays often bypass conventional plagiarism detection systems and threaten the credibility of student assessments. To address this challenge, we propose XLNet-CNN, a hybrid detection framework that combines XLNet (Generalized Autoregressive Pretraining for Language Understanding) with a convolutional neural network (CNN) for local feature extraction. Using the EnglishQA Essays Corpus containing 161,640 essays from both human and AI sources, we benchmark the model against representative machine learning, deep learning, and transformer-based baselines, as well as widely used external detection tools. The framework achieves accuracy 0.98, recall 0.96, F2 score 0.97, precision 1.00, BLEU (Bilingual Evaluation Understudy) 30.76, and perplexity (PPL) 3.99. Notably, it eliminates false negatives, ensuring that no AI-generated essay is misclassified as human-written, a critical safeguard for high-stakes academic contexts. These results demonstrate the effectiveness of hybrid architectures in capturing both semantic coherence and stylistic artifacts, offering a reliable and scalable solution for maintaining fairness in academic evaluation while supporting responsible AI use in education.

Keywords AI-generated essays · Academic integrity · Text Detection · NLP tasks · Transformer · XLNet-CNN · Deep learning

1 Introduction

In recent years, large language models (LLMs) have advanced significantly, demonstrating the ability to generate text that closely resembles human writing. This progress has introduced new challenges in education, particularly with respect to academic integrity. A central concern is the growing use of LLMs by students to generate essays that appear authentic, thereby undermining the credibility of academic assessments. Since the release of ChatGPT, generative AI has become a mainstream phenomenon, attracting

widespread adoption and raising urgent concerns about plagiarism and academic dishonesty [1, 2].

Evidence suggests that existing plagiarism detection systems are insufficient for this challenge. Yeadon [3] found that AI-generated short essays received very low plagiarism detection scores (1–2%) from widely used tools such as Turnitin and Grammarly. Some researchers even argue that efforts to distinguish AI- from human-generated text may be futile [4]. Yet, with universities continuing to rely on plagiarism detection rather than dedicated AI-detection frameworks, academic institutions face increasing risks to credibility and fairness [3].

Recent advancements in natural language processing (NLP), particularly with models such as GPT-3 and GPT-4 [5, 6], have been driven by unsupervised representation learning. Autoregressive (AR) models, which predict the next token in a sequence, and autoencoding (AE) models, which leverage masked token prediction, have provided high-quality pretrained embeddings. These developments have laid the groundwork for building robust detection models. A variety of machine learning and deep learning

✉ Santos Kumar Baliarsingh
santos.baliarsingh@kiit.ac.in

Manish Prajapati
2381040@kiit.ac.in

Prabhu Prasad Dev
prabhu.dev@kiit.ac.in

¹ School of Computer Engineering, KIIT Deemed to be University, Patia, Bhubaneswar 751024, Odisha, India

approaches have also been investigated for related classification tasks, including SVMs [7], CNNs [8], RNNs [9], LSTMs [10], BiLSTMs [11], and GRUs [12]. Hybrid methods such as CNN-LSTM [13] and CNN-Attention-LSTM [14] combine local feature extraction with sequential modeling, while pretrained Transformers, including BERT, RoBERTa [15], and XLNet [16] further enhance contextual understanding.

Despite these advances, detecting AI-generated text remains a difficult problem. Traditional plagiarism tools are not robust to rapidly evolving LLMs, and many detection systems suffer from high false-positive rates, unfairly penalizing genuine student work. This gap highlights the urgent need for methods that can capture both the statistical artifacts of machine-generated text and the nuanced semantic structures of human writing.

Motivated by this challenge, we explore a hybrid architecture that integrates XLNet's autoregressive pretraining and bidirectional context modeling with the convolutional neural network's ability to extract local features. The XLNet-CNN framework aims to provide educators with a reliable detection system that balances accuracy with fairness. By equipping institutions with tools that distinguish between student-written and AI-generated essays, our work seeks to preserve academic integrity while acknowledging the constructive potential of LLMs in education.

Our main contributions are as follows:

- We propose XLNet-CNN, a hybrid model that combines XLNet's pretrained contextual embeddings with CNN and LSTM layers.
- The model captures both long-range dependencies and local textual features for improved detection accuracy.
- We benchmark XLNet-CNN against widely used AI-detection tools (OpenAI Detector, GPTZero, DetectGPT, Copyleaks, Grammarly, Turnitin).
- Performance is evaluated using precision, recall, and F2 score, emphasizing both accuracy and fairness.
- Our framework provides a transparent, practical, and adaptable solution to help educators distinguish between AI-generated and student-written essays.

The remainder of this paper is organized as follows: Sect. 2 reviews related literature, Sect. 4.1 describes the dataset, Sect. 3 presents the methodology, Sect. 4 discusses results and experimental setup, Sect. 5 outlines challenges and limitations, and Sect. 6 concludes with future research directions.

2 Related work

Research on detecting AI-generated text has grown rapidly with the rise of large language models (LLMs). Two influential studies shaped our approach: Gehrmann et al. [17] and Sadasivan et al. [18], which highlighted both statistical methods and practical challenges in detecting AI-written essays.

2.1 Development of language model

The evolution of language models has significantly transformed the field of NLP, enabling the extraction of meaningful insights from vast amounts of unstructured text. The evolution of NLP has been marked by significant milestones, transforming the way machines understand and interpret human language. Initially, NLP models relied on probabilistic approaches, focusing on learning word occurrence probabilities from vast document collections [19]. However, the introduction of Word2Vec in 2013 revolutionized the field by converting text into word embeddings, enabling various NLP tasks such as identifying semantically similar words [20]. Despite its groundbreaking impact, Word2Vec's limitations in addressing polysemy and capturing contextual nuances became apparent, prompting further innovations [21].

In order to get over these restrictions, scientists looked into pretraining techniques and directed language models, which provide context-based dynamic word representations. This shift enabled more accurate and context-aware language understanding, as seen in the works [22, 23]. These advancements have paved the way for more sophisticated NLP models, capable of capturing the nuances of human language. A significant milestone in this evolution was the development of ELMo, which used a bidirectional LSTM (biLSTM) network to produce context-aware word representations, thus enhancing the handling of word meanings within different contexts [22]. The introduction of the Transformer architecture in 2017 [24] marked a breakthrough. This architecture, with its attention mechanism, excelled in managing long-term dependencies more effectively than LSTMs. Building upon this foundation, the Bidirectional Encoder Representations from Transformers (BERT) model was introduced in 2018 [25]. BERT's bidirectional encoding approach significantly improved the model's understanding of context and semantics.

The landscape of NLP has undergone a significant transformation with the advent of the Generative Pre-trained Transformer (GPT) model, introduced by OpenAI [26]. This pioneering architecture, focusing on the decoder component of the Transformer, has revolutionized the field by establishing new benchmarks and catalyzing the "pre-training and

fine-tuning" paradigm. This paradigm has inspired a new generation of models, including BART [27], RoBERTa [28], and T5 [29], which have further refined the pre-training and fine-tuning approach, expanding its applicability to a wide range of tasks such as question answering, sentiment analysis, and named entity recognition. Recent breakthroughs in computational power and the availability of large datasets have given rise to LLMs, marking a significant departure from traditional language models [30]. Notably, GPT-3 [31] and GPT-4 [32] have demonstrated remarkable capabilities, redefining the interaction between AI and human users. ChatGPT, built on GPT-3 and GPT-4, has exhibited exceptional conversational abilities, setting a new standard for AI-driven dialogue systems. In contrast, LLaMA [33] has focused on enhancing efficiency and reducing resource consumption, making it an attractive option for diverse text generation tasks.

2.2 Evolution of prompting techniques in ChatGPT

Supervised learning has long been a staple in NLP, yet it heavily depends on having a sufficiently labeled dataset. The emergence of prompt engineering, also known as prompt learning, has revolutionized the field of NLP by providing an innovative solution to the limitations imposed by traditional fine-tuning methods. This approach leverages the capabilities of LLMs, such as ChatGPT, to perform various NLP tasks without the need for extensive labeled data or gradient updates. By crafting specific task descriptions, known as prompts, researchers can guide the model to achieve remarkable results in zero-shot or few-shot settings [34–36]. ChatGPT, in particular, has been extensively explored for its potential in various domains, including healthcare and medical fields, due to its user-friendly interface and impressive ability to generate human-like text. Studies have demonstrated its effectiveness in scientific writing [37, 38], research topic generation [39, 40], and domain report translation [41]. However, despite its promise, challenges such as hallucinations, plagiarism, and research fabrication highlight the need for rigorous human oversight, careful consideration, and regulation [42, 43].

2.3 XLNet modeling method

The permutation-based AR modeling has been studied before in [44, 45], although with several important differences in emphasis. XLNet is intended to allow AR language models to retain bidirectional context, while previous models concentrated on improving density estimation by incorporating an orderless inductive bias into the model. According to the technical details, XLNet does this by explicitly integrating the target location into the hidden

state using a two-stream attention method, unlike previous permutation-based AR models that relied on implicit positional awareness in their MLP structures. Notably, for both orderless NADE and XLNet, the term orderless does not mean that the input sequence is randomly permuted; rather, it indicates that the model is capable of supporting various factorization orders for the distribution. Autoregressive denoising is a comparable idea in text generation [46], but it only acts in a given order.

2.4 Pre-trained embeddings and hybrid approaches

Early advances in NLP focused on pre-trained embeddings that capture semantic relationships between words. Word2Vec [20] represented words in a continuous vector space, where semantic similarity corresponded to spatial proximity. This inspired hybrid models such as Word2Vec + CNN [47], which improved feature extraction, and LSTM-based models that integrated Word2Vec to better handle sequential dependencies. GloVe [48] extended this line of work by combining co-occurrence statistics with global vector representations, yielding compact and semantically rich embeddings. However, both Word2Vec and GloVe remain static, unable to handle polysemy or adapt to contextual variation [49, 50]. To address these limitations, recent research has integrated embeddings into hybrid architectures. XLNet [16], building on the Transformer framework [25], introduced a permutation-based autoregressive objective with two-stream attention. This design enables dynamic, context-sensitive representations while maintaining the strengths of autoregressive modeling. By bridging static embeddings and contextual modeling, such hybrid approaches demonstrate strong adaptability and are particularly useful for detecting subtle differences between human- and AI-generated text.

3 Methodology

3.1 XLNet model

XLNet is a state-of-the-art NLP model that builds upon the Transformer architecture, introducing a generalized autoregressive pretraining method for language understanding. It addresses the limitations of previous models, such as BERT, by integrating bidirectional context learning with autoregressive pretraining, making it particularly effective for tasks like human and AI-generated essays detection. Detecting whether text has been generated by AI or written by a human has become increasingly relevant as large language models like GPT-3 and GPT-4 become more advanced. Traditional models like BERT provided strong contextual learning but had inherent limitations in handling autoregressive

tasks, which are crucial for detecting nuanced differences in text. XLNet addresses these gaps by combining the strengths of autoregressive language modeling with bidirectional context learning, making it a powerful tool for AI and human-generated text detection.

The core idea behind XLNet is to combine the strengths of AR language modeling and AE methods. While models like BERT use AE for learning bidirectional contexts, XLNet incorporates autoregressive features to model language by considering all permutations of the factorization order of input tokens.

3.1.1 Autoregressive language modeling

In the context of detecting AI-generated essays, we compare Autoregressive (AR) Language Modeling with BERT-style pretraining, as illustrated in the Figure. AR-based models (such as GPT) rely on a unidirectional probability distribution, where the probability of a token y_t is conditioned on the preceding words. This can be mathematically represented as (see the Eq. 1):

$$\max_{\phi} \sum_{t=1}^T \log q_{\phi}(y_t | y_{<t}) = \sum_{t=1}^T \log \frac{\exp(g_{\phi}(y_{1:t-1})^T v(y_t))}{\sum_{y'} \exp(g_{\phi}(y_{1:t-1})^T v(y'))} \quad (1)$$

where $g_{\phi}(y_{1:t-1})$ is the hidden representation of the preceding sequence, and $v(y)$ represents the token embedding. For AI-generated essays detection, AR models learn sequential dependencies, capturing statistical anomalies that often characterize AI-generated essays. These include repetitive patterns, unnatural token transitions, and over-reliance on high-frequency n-grams key indicators of AI-generated essays.

Conversely, BERT-based detection models use MLM, where some tokens are randomly masked and then predicted based on bidirectional context. This method is described mathematically as (see Eq. 2):

$$\max_{\phi} \sum_{t=1}^T z_t \log q_{\phi}(y_t | \tilde{y}) = \sum_{t=1}^T z_t \log \frac{\exp(G_{\phi}(\tilde{y})^T v(y_t))}{\sum_{y'} \exp(G_{\phi}(\tilde{y})^T v(y'))} \quad (2)$$

where $z_t = 1$ if y_t is masked, and $G_{\phi}(\tilde{y})$ represents the learned hidden representation of the masked sequence $\tilde{y}$. Unlike AR models, BERT-style detectors leverage bidirectional context, making them effective at recognizing contextual inconsistencies and detecting subtle linguistic artifacts in AI-generated essays. Since LLMs often generate text that appears fluent but lacks deep discourse coherence, BERT-based models can highlight semantic and syntactic inconsistencies that differentiate AI-generated text from human-written essays.

A major challenge in AI-text detection is handling inference-time discrepancies. BERT models introduce artificial [MASK] tokens, making the training environment different from real-world classification scenarios. On the other hand, AR models predict the next word naturally but struggle to capture global coherence. A hybrid detection approach combining AR-based sequential dependency learning with BERT-style bidirectional context modeling can improve classification accuracy by leveraging both local statistical cues and broader linguistic structures present in AI-generated essays.

3.1.2 Permutation language modeling

XLNet presents a permutation-based training objective in contrast to models such as BERT, which employ a masked language modeling (MLM) objective. Maximizing the chance of a sequence under every feasible variation of the token order is the main concept. Given an input sequence $x = (x_1, x_2, \dots, x_n)$, XLNet considers a random permutation of the sequence $z = (z_1, z_2, \dots, z_n)$, where each z_i represents a reordering of tokens. The probability of each token x_{z_t} is conditioned on its preceding tokens in the permutation (see Eq. 3):

$$P(x_{z_t} | x_{z_1}, \dots, x_{z_{t-1}}) \quad (3)$$

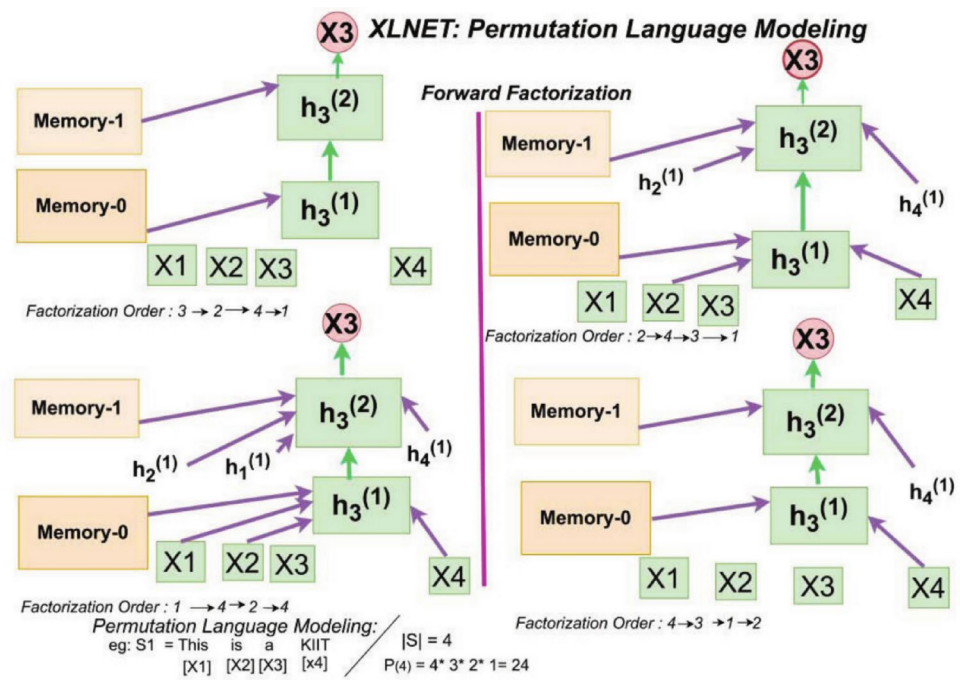
This objective enables XLNet to capture dependencies in multiple directions, giving it a richer understanding of context a critical aspect for detecting subtle inconsistencies in AI-generated essays compared to human writing.

Permutation language modeling is a technique used by XLNet, a state-of-the-art pre-trained language model. This approach is different from traditional autoregressive models, such as GPT, which predict the next word in a sequence based on the previous words. Instead, XLNet generates predictions by considering all possible permutations of the input sequence, allowing it to learn bidirectional context while maintaining autoregressive concepts.

Figure 1 illustrates XLNet's Permutation Language Modeling, where multiple permutations of input tokens are created to predict the context. It incorporates memory states for better context representation. For example, given the sequence $S = [X1, X2, X3, X4]$, permutations like $[3 \rightarrow 2 \rightarrow 4 \rightarrow 1]$ are modeled. Each token ($X3$) generates hidden states ($h_3^{(1)}, h_3^{(2)}$) based on memory and surrounding tokens. This bidirectional approach enhances understanding by learning dependencies from all directions, improving over traditional language modeling.

Key concept: In permutation language Modeling, the model doesn't strictly follow a left-to-right or right-to-left

Fig. 1 The illustration depicts the XLNet Permutation Language Modeling approach, showcasing forward factorization and multiple factorization orders. The permutation of token sequences enables bidirectional context learning by modeling all possible factorization orders (3 → 2 → 4 → 1, 2 → 4 → 3 → 1, etc.). Each factorization order processes token dependencies using memory layers (Memory-0, Memory-1) and computes hidden states ($h_t^{(1)}$, $h_t^{(2)}$) for a target token (X_3). The example demonstrates how this mechanism allows XLNet to leverage the benefits of both autoregressive models and bidirectional contexts for enhanced performance in natural language understanding tasks



order of word predictions. Equation 4 Instead, it randomly permutes the order in which the words are predicted, allowing the model to learn dependencies across the entire sequence. This approach leads to a deeper understanding of context because the model learns not just to predict the next word, but also to understand relationships between words that appear at different positions within the sequence. For example, consider a sentence like "This is a KIIT." XLNet might randomly permute the positions of the words in the sequence and then predict each word based on its surrounding context. The number of possible permutations is calculated as the factorial of the sequence length (for a sentence with four words, the number of permutations is $4! = 24$).

Why permutation?: The advantage of permutation over a fixed order (like left-to-right in GPT) is that it allows XLNet to capture more diverse contextual relationships in the text. This model retains the benefits of autoregressive models, where each word is conditioned on others, but it also incorporates the strength of bidirectional models (like BERT), where the context of each word can come from both its left and right.

For a sentence of length S , the number of permutations $P(S)$ is computed as (see Eq. 4):

$$P(S) = S! = S \times (S - 1) \times (S - 2) \times \dots \times 1 \quad (4)$$

For example, if $S = 4$, the number of possible permutations is $P(4) = 4 \times 3 \times 2 \times 1 = 24$.

Impact on Model Performance: This permutation-based approach allows XLNet to capture rich, context-aware representations of language, which often results in better

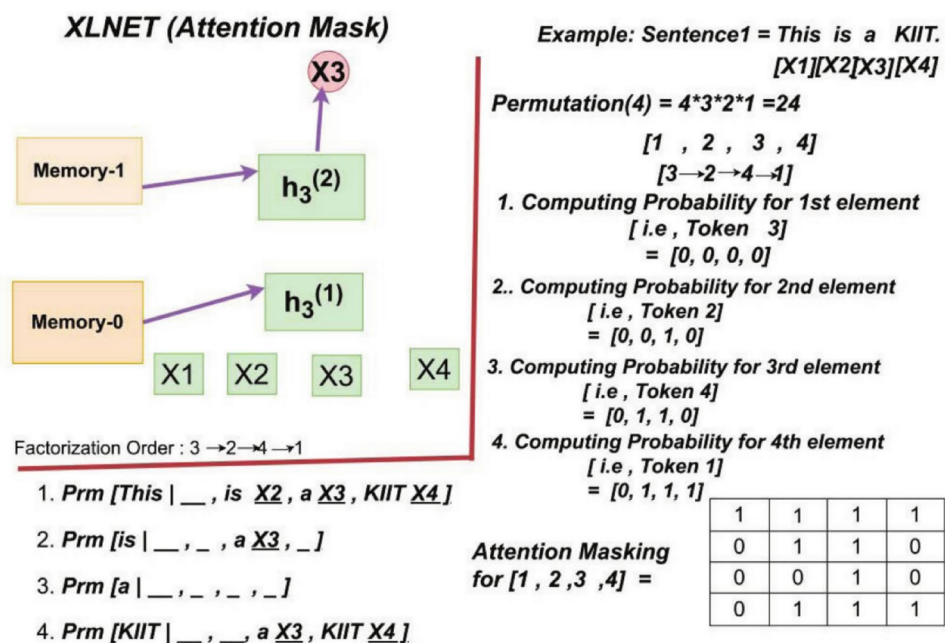
performance on various NLP tasks like text classification, question answering, and, importantly, essay generation and detection. It balances the flexibility of autoregressive models with the bidirectionality of models like BERT, making XLNet particularly effective for complex language tasks.

3.1.3 XLNet: attention masks

XLNet uses an attention mask to control which tokens the model should attend to in a sequence, especially for padding tokens or any other tokens that should not contribute to the model's understanding. The attention mask is particularly useful in tasks like essay generation, where you might need to focus on certain parts of the text (like the essay content) and ignore irrelevant parts (like padding or masked tokens).

Let's walk through an example of how an attention mask works in the context of detecting AI-generated essays versus human-generated essays using XLNet. XLNet uses the attention mechanism to weigh the importance of different tokens in the input sequence. The attention mask plays a crucial role in controlling which tokens the model should focus on while processing input sequences. This is especially important when working with padded sequences. Figure 2 illustrates XLNet's Attention Masking mechanism. For a sentence like "This is a KIIT," permutations of token order (e.g., $[3 \leftarrow 2 \leftarrow 4 \leftarrow 1]$) are used to compute probabilities sequentially. Each token can only attend to previous tokens in the permutation order. Attention masks ensure this causality by marking which tokens can be attended (e.g., $[0, 0, 1, 0]$ for Token 2). The mask matrix enforces this rule

Fig. 2 XLNet's Attention Masking mechanism enables bidirectional context learning by enforcing causal dependencies through permutation-based factorization orders. The attention mask matrix ensures each token attends only to permissible tokens in the given permutation, preserving context integrity during computation



across the entire sequence, enabling bidirectional context learning without data leakage.

In XLNet, the attention mask is a binary tensor that indicates which tokens should be attended to and which should be ignored. It typically has a shape of (batch_size, sequence_length), where each position holds a value of 1 if the corresponding token is real (not padding) and 0 if it's padding. The attention mask ensures that padding tokens do not interfere with the model's ability to focus on the actual content of the input text.

For example: if you have the sequence "This is a KIIT" and pad it to a length of 10 with zero-padding, the attention mask would look like this: [1, 1, 1, 1, 0, 0, 0, 0, 0, 0] Here, 1 indicates a token the model should pay attention to, and 0 indicates a padding token to ignore. The attention mask ensures that only the meaningful tokens are attended to during the computation of attention scores, avoiding bias from the padding.

Remark about Permutation: the objective proposed alters only the factorization order while maintaining the sequence order intact. This ensures the preservation of the original sequence order, utilizing the corresponding positional encodings for that order. A suitable attention mask within Transformers guarantees the proper permutation of the factorization order. This method is essential as the model only sees the text patterns in their natural order when it is being fine-tuned.

The token z_3 may be predicted using the same input sequence z , but with different factorization ordering, as shown in Fig. 1.

Although the permutation language modeling objective has beneficial characteristics, a simple implementation with a conventional Transformer setup may not perform efficiently. To illustrate the problem, consider modeling the

next token distribution $p_\theta(Y_{x_t} | Y_{x_{<t}})$ using the typical softmax approach (see Eq. 5):

$$p_\theta(Y_{x_t} = y | Y_{x_{<t}}) = \frac{\exp(h(y)^\top p_\theta(Y_{x_{<t}}))}{\sum_{y'} \exp(h(y')^\top p_\theta(Y_{x_{<t}}))}, \quad (5)$$

where the hidden state of $Y_{x_{<t}}$ produced by the shared Transformer network with appropriate masking is represented by $p_\theta(Y_{x_{<t}})$.

Observe that the target location x_t has no bearing on $p_\theta(Y_{x_{<t}})$. This hinders the learning of meaningful representations since the same distribution is anticipated regardless of the target position.

In order to get around this, we suggest adjusting the next token distribution to take the intended position into consideration (see Eq. 6):

$$p_\theta(Y_{x_t} = y | Y_{x_{<t}}) = \frac{\exp(h(y)^\top n_\theta(Y_{x_{<t}}, x_t))}{\sum_{y'} \exp(h(y')^\top n_\theta(Y_{x_{<t}}, x_t))}, \quad (6)$$

where $n_\theta(Y_{x_{<t}}, x_t)$ introduces a new type of representation that integrates the target position x_t as input.

Two streams of self-attention: it is difficult to define $n_\theta(Y_{x_{<t}}, x_t)$, even though the idea of target-aware representations helps to clear up the ambiguity in target prediction. Out of all the options, we suggest that we "stand" in the desired location x_t and use it to collect information from the context $Y_{x_{<t}}$ by paying attention to everything.

There are two requirements that must be met for this parameterization to function properly:

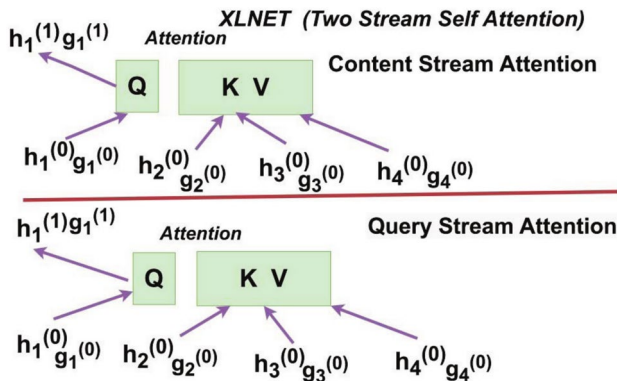


Fig. 3 Content stream attention, identical to standard self-attention. (c): Query stream attention, which lacks access to information about the content x_{yt}

- The target position x_t should be the sole thing used to forecast the token Y_{x_t} , not the content $Y_{x_{<t}}$; otherwise, the task becomes simplistic.
- The content $Y_{x_{<t}}$ must be included in order to forecast other tokens Y_{x_j} for $j > t$, $n_\theta(Y_{x_{<t}}, x_t)$ and guarantee full contextual information.

$$q_{xt}^{(n)} \leftarrow \text{Self Attention}(Q = q_{xt}^{(n-1)}, KV = p_{x_{<t}}^{(n-1)}; \theta), \quad (\text{query stream: uses } x_t \text{ but cannot access } y_{x_t})$$

$$p_{xt}^{(n)} \leftarrow \text{Self Attention}(Q = p_{xt}^{(n-1)}, KV = p_{x_{<t}}^{(n-1)}; \theta), \quad (\text{content-stream: uses both } x_t \text{ and } y_{x_t}).$$

Here, the query, key, and value in the attention operation are represented by Q, K, V . Similar to the ordinary self-attention procedure, the update rule applies to content representations. The content stream may therefore be used as a conventional Transformer model for fine-tuning, and the

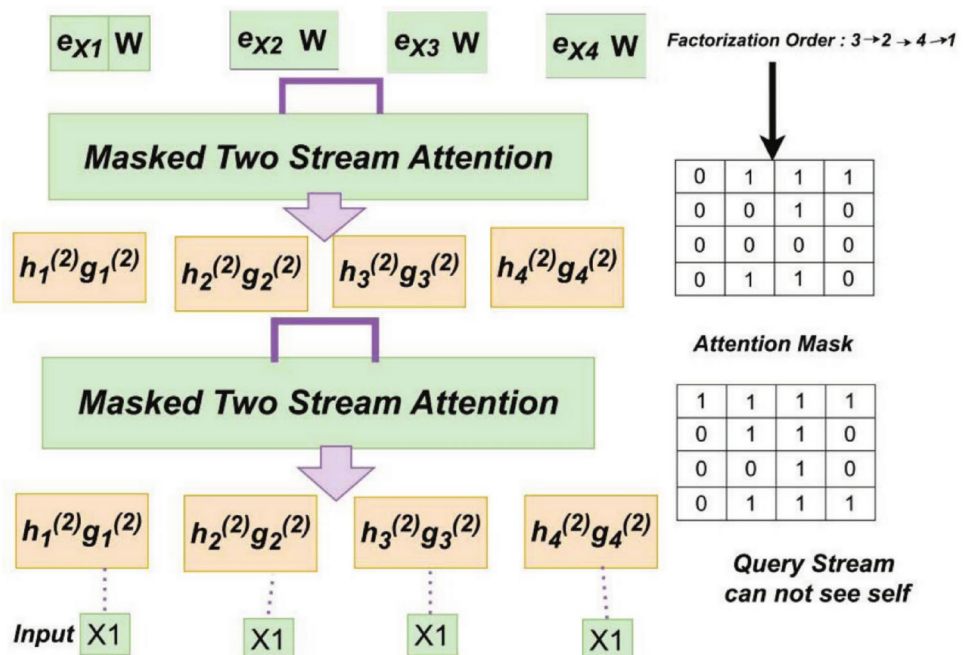
To resolve this conflict, we introduce two distinct sets of hidden-representations:

- The function of content-representation $p_\theta(Y_{x_{<t}})$, represented as p_{x_t} , is comparable to that of the typical hidden states in Transformers. This representation encodes $Y_{x_{<t}}$ as well as the context.
- Query-representation $n_\theta(Y_{x_{<t}}, x_t)$, denoted as n_{x_t} , relies only on the context $Y_{x_{<t}}$ and the target position x_t , without directly accessing the content $Y_{x_{<t}}$.

Computationally speaking, the content stream is initialized from the corresponding word embeddings, i.e., $p_j^{(0)} = g(y_j)$, whereas the first layer query stream is initialized with a trainable vectors, i.e., $q_j^{(0)} = v$. For every self-attention layer $n = 1, \dots, N$, the two streams of representations undergo iterative updates in the manner described below:

Using a shared set of parameters, the representation streams evolve as shown in Fig. 3 and Fig. 4.

Fig. 4 XLNet's masked two-stream attention mechanism. The content stream ('h') encodes token representations, while the query stream ('g') maintains autoregressive properties. The factorization order (e.g., '3 $\leftarrow$ 2 $\leftarrow$ 4 $\leftarrow$ 1') determines token processing. The attention mask ensures the query stream avoids self-attention, preserving causality. This design enables XLNet to leverage bidirectional context effectively while retaining the benefits of autoregressive modeling for improved language understanding and generation



Partially predicted: with standing the obvious advantages of the permutation language modeling goal objective (Eq. 3) has several benefits, the complexity of the permutations makes it an challenging optimization issue, which causes first trials to converge slowly. In order to address this problem, we limit predictions to the last tokens in a specified factorization order.

Specifically, y is divided into a target subsequence $y_{>d}$ and a non-target subsequence $y_{\leq d}$, where d is the cutoff point. The objective is to maximize the target subsequence's log-likelihood conditioned on the non-target subsequence (see Eq. 7):

$$\max_{\theta} \mathbb{E}_{y \sim \mathcal{Y}_T} \left[\sum_{t=d+1}^{|y|} \log p_{\theta}(w_{y_t} \mid w_{y_{\leq d}}) \right]. \quad (7)$$

Here, $y_{>d}$ is chosen as the target, as it incorporates the longest context within the sequence according to the current factorization order y .

To optimize the model's efficiency, a hyperparameter L is introduced such that approximately $1/L$ tokens are selected for prediction, i.e., $|y|/(|y| - d) \approx L$. For tokens that are not selected, their query representations are omitted, which leads to gains in both processing speed and memory utilization.

3.2 CNN model

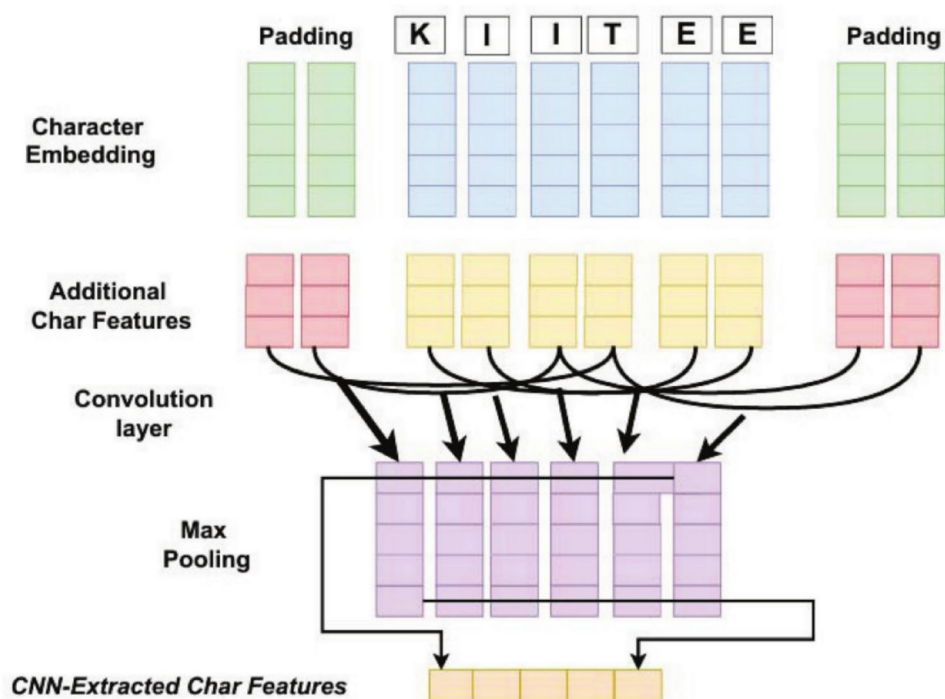
Convolutional Neural Networks (CNNs), originally designed for image processing, have proven to be highly effective in text classification tasks, including the detection of AI-generated text. Their strength lies in their ability to automatically extract local and hierarchical features from sequences of text, enabling the model to recognize patterns such as word n-grams, syntactic structures, and stylistic cues that differentiate human-written content from machine-generated text (see the Fig. 5).

3.2.1 Character embedding and feature representation

The process begins with character-level embedding, where each character in a text is converted into a dense vector representation. As shown in the diagram, characters such as "K", "I", "I", "T", "E", "E" are mapped into embedding vectors. This representation captures semantic and orthographic information at the most granular level, which is especially useful for handling misspellings, rare words, and stylistic variations commonly observed in both human and AI-generated text. Padding is applied to ensure consistent sequence lengths for convolution operations.

In addition to embeddings, auxiliary character features can be included. These features may capture statistical, syntactic, or frequency-based information that complements the learned embeddings. Together, embeddings and additional features form the initial representation that is fed into the CNN.

Fig. 5 Pipeline of a CNN model for detecting AI-generated text. Character embeddings are processed through convolution and max pooling layers to extract salient features, which are then used for classification



3.2.2 Convolutional layers for local feature extraction

The convolution layer applies multiple filters across the embeddings, sliding over the input sequence to capture local dependencies and patterns. For instance, a filter of width three might detect common trigrams, while larger filters can capture longer phrases. Each filter produces a feature map that highlights the presence of specific n-gram features in the text. This is crucial in AI-generated text detection, as generative models often leave subtle traces in local word patterns, repetitions, or unnatural phrase constructions.

By stacking multiple convolutional filters, the model can learn a diverse set of features ranging from simple character combinations to complex syntactic structures. These learned patterns serve as discriminative signals to differentiate between authentic human writing and AI-generated outputs.

3.2.3 Max Pooling for Dimensionality Reduction

After convolution, max pooling is applied to reduce the dimensionality of the feature maps while retaining the most informative signals. This process condenses variable-length inputs into fixed-size vectors, making the representation more manageable for downstream layers. Importantly, max pooling ensures that the model focuses on the most salient features, such as unusual word sequences or repetitive structures, which are often indicative of machine-generated text.

The resulting pooled feature vectors are referred to as CNN-extracted character features. These compact,

information-rich vectors are highly effective for classification tasks.

3.2.4 Classification layer and detection

The extracted features are then passed to one or more dense layers, culminating in a classification layer (often with a sigmoid or softmax activation) that predicts whether the input text is human-written or AI-generated. During training, the model learns to assign higher weights to discriminative patterns such as coherence, grammar consistency, and semantic flow, which are typically more natural in human text but may appear mechanical or inconsistent in AI-generated outputs.

3.3 XLNet-CNN hybrid model

The increasing use of LLMs, such as GPT-3 and GPT-4, has raised significant concerns in academic settings regarding the authenticity of student work. Detecting AI-generated essays is crucial for maintaining academic integrity and ensuring the validity of assessments. To address this challenge, this paper presents the XLNet-CNN Algorithm 1 and , a hybrid approach that combines the XLNet transformer model with CNNs for detecting human and AI-generated essays, as shown in Fig. 6. This model is evaluated using the "EnglishQA Essays Corpus", which contains Question(N) = 161,640 (212,479 human-generated; 169,668 AI-generated essays, samples labeled as either human-written or AI-generated.

```

1: function CREATE_MODEL
2:   Input: Token IDs and attention mask of shape (256,)
3:   Output: Compiled neural network model
4:   Print("Loading XLNetModel")
5:   Load pre-trained XLNet model xlnetModel
6:   Define shared Conv1D layer: conv1D_shared  $\leftarrow$  Conv1D(64, kernel_size = 7, strides =
7:     2, regularization = L2(0.01))
8:   Define batch normalization layer: batchN  $\leftarrow$  BatchNormalization()
9:   Define activation function: activa  $\leftarrow$  Activation(relu)
10:  Pass input through XLNet model:
11:  xlnetout  $\leftarrow$  xlnetModel.transformer({input_ids, attention_mask})
12:  Extract last hidden state: x  $\leftarrow$  xlnetout.last_hidden_state
13:  Apply Conv1D layer: x  $\leftarrow$  conv1D_shared(x)
14:  Apply batch normalization: x  $\leftarrow$  batchN(x)
15:  Apply activation function: x  $\leftarrow$  activa(x)
16:  Apply max pooling: x  $\leftarrow$  MaxPooling1D(3, strides = 2)(x)
17:  Flatten output: x  $\leftarrow$  Flatten()(x)
18:  Apply dense layer with sigmoid activation: x  $\leftarrow$  Dense(1, activation = sigmoid)(x)
19:  Apply dropout: x  $\leftarrow$  Dropout(0.3)(x)
20:  Define model: model  $\leftarrow$  Model(inputs = [input_ids, attention_mask], outputs = x)
21:  Print model summary
22:  Define optimizer: adam  $\leftarrow$  Adam(learning_rate = 0.00001)
23:  Compile model with binary cross-entropy loss and accuracy, precision, and recall metrics:
24:  model.compile(optimizer = adam, loss = BinaryCrossentropy(from_logits = False),
25:    metrics = [accuracy, Precision(), Recall()])
26:  return model
27: end function

```

Algorithm 1 Create XLNet-CNN-based model

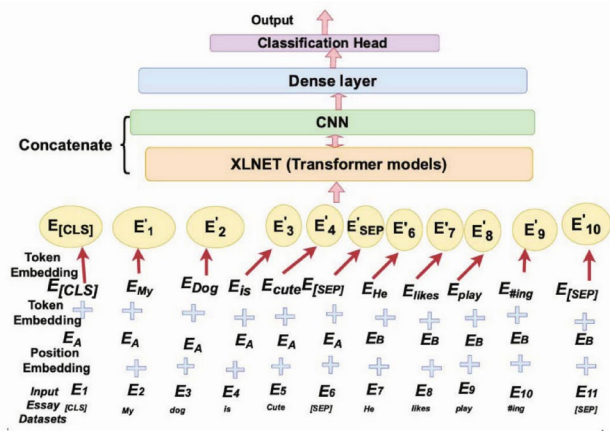


Fig. 6 Proposed methodology: XLNet-CNN hybrid model for AI-generated essays detection

The XLNet-CNN algorithm combines XLNet's power of generalized autoregressive pretraining with the ability of CNNs to capture local patterns and structures in text. This hybrid model effectively utilizes XLNet's bidirectional context learning while capturing distinctive stylistic features that can differentiate human-written text from AI-generated essays.

This paper provides a detailed explanation of the XLNet-CNN algorithm, the steps involved in training and testing the model, and the role of word embeddings in the process. The following sections break down the methodology, data preprocessing, model architecture, training, and fine-tuning processes.

3.3.1 XLNet-CNN model architecture overview

The XLNet-CNN model integrates two key components: XLNet: A transformer model that learns bidirectional dependencies between tokens, leveraging generalized autoregressive pretraining. CNNs: Used for feature extraction and capturing local patterns within the text.

XLNet is a transformer-based model designed to overcome limitations of traditional bidirectional transformers like BERT. While BERT relies on a masked language model (MLM) to predict missing tokens, XLNet employs a generalized autoregressive model. Instead of masking tokens, XLNet learns to predict tokens in any possible order, thereby capturing bidirectional context while also benefiting from autoregressive capabilities. Mathematically, the training objective for XLNet is (see Eq. 8):

$$P_{\theta}(x) = \prod_{i=1}^n P_{\theta}(x_i | x_{1:i-1}) \quad (8)$$

Where: - x_i represents the tokens in a sequence; - $P_{\theta}(x_i | x_{1:i-1})$ is the probability of predicting the i^{th} token given all the previous tokens in the sequence; - θ are the model parameters; - n is the sequence length.

This training paradigm allows XLNet to capture richer semantic relationships and bidirectional dependencies, making it ideal for complex NLP tasks like human vs. AI-generated essays classification.

3.3.2 CNN: convolutional layers for feature extraction

After XLNet generates token embeddings, these embeddings are passed through a series of convolutional layers. CNNs are well-known for their ability to capture local patterns and structures in data. In the case of text, CNNs learn to identify characteristic features such as sentence structures, word usage patterns, or stylistic elements typical of human or AI-generated essays.

A convolutional layer applies a filter over the input data to detect patterns. The mathematical operation for the convolution is (see Eq. 9):

$$y_j = \max \left(\sum_{i=1}^m w_i x_{i+j-1} + b \right) \quad (9)$$

Where: - y_j is the output at position j , - w_i are the weights (filters); - x_{i+j-1} are the input values; - b is the bias term; - m is the size of the filter.

This process is applied repeatedly with different filters, resulting in a set of feature maps that highlight important aspects of the text that may distinguish human from AI-generated essays.

3.3.3 Preprocessing for the englishQA essays corpus

The englishQA essays corpus consists of 161,640 essays, each labeled as human-written or AI-generated. Data preprocessing is a critical step to ensure the model receives clean, relevant input. The preprocessing pipeline consists of the following steps:

Text normalization and tokenization

- **Lowercasing:** All text is converted to lowercase to reduce variations in the data caused by capitalization.
- **Tokenization:** SentencePiece, a subword tokenization method, is applied to split the text into smaller units, or tokens. This approach helps the model handle rare or out-of-vocabulary words more effectively.
- **Stop-word Removal:** Common words (such as "the," "a," and "and") that do not carry significant meaning are removed from the text to reduce noise and focus on important content.
- **Punctuation Removal:** Non-informative punctuation marks (commas, periods, etc.) are eliminated to avoid unnecessary complexity.

- **Digit Removal:** Numbers are removed to prevent irrelevant noise, as they do not add meaningful information for distinguishing between human and AI text.
- **Whitespace and Noise Removal:** Any extra spaces, special characters, or HTML tags are stripped from the text to ensure clean data input.

Embedding generation: XLNet tokenization and embeddings after preprocessing, the text is tokenized into subword units using XLNet's pretrained tokenizer. These tokens are then transformed into dense vector representations (embeddings), which capture the semantic meaning of the words and their relationships in context. XLNet embeddings encode both local and global context information, which is critical for detecting AI-generated essays.

3.3.4 Model training and fine-tuning

The XLNet-CNN model is trained on the preprocessed EnglishQA Essays Corpus dataset. The dataset is split into training (60%), validation (20%), and testing (20%) sets. The training process follows standard supervised learning principles, where the model learns to distinguish between human-written and AI-generated essays. Once the data is preprocessed and the embeddings are generated, the model enters the training and fine-tuning phases.

Stepwise training process:

1. **Input representation:** The text is tokenized and fed into XLNet. Each token is represented by a dense embedding vector generated by XLNet's transformer-based architecture.
2. **Feature Extraction with CNN:** The embeddings are passed through a series of convolutional layers. These layers apply filters to detect local features, such as syntax or grammar patterns, that are indicative of either human-written or AI-generated essays.
3. **Max Pooling Layer:** After the convolutional layers, a max-pooling operation is applied to reduce the dimensionality of the feature maps while retaining the most important features. The max-pooling operation selects the most informative values from each feature map.
4. **Fully connected Layer:** The pooled features are passed through one or more fully connected layers. These layers consolidate the information extracted by the CNN and XLNet and help the model learn complex relationships between the features.
5. **Output Layer:** A softmax activation function is applied to the final output layer, which produces a probability distribution over the two classes: human-written and AI-generated.

```

1: function XLNET-CNN-TEXT-DETECTION
2:   Input: Text samples  $\mathcal{T}$  with tokenized representations
3:   Output: Binary classification (Human or AI-generated)
4:   Step 1: Preprocessing
5:     Tokenize input text samples  $\mathcal{T}$ 
6:     Convert tokens to input IDs and attention masks
7:   Step 2: XLNet Feature Extraction
8:     Load pre-trained XLNet model
9:     Pass input IDs and attention masks to XLNet
10:    Extract last hidden state representation:  $H = XLNet(T)$ 
11:  Step 3: CNN-based Feature Refinement
12:    Define 1D Convolutional layer:  $Conv1D(filters = 64, kernel\_size = 7, strides = 2)$ 
13:    Apply Conv1D to XLNet output:  $X = Conv1D(H)$ 
14:    Apply batch normalization:  $X = BatchNorm(X)$ 
15:    Apply ReLU activation:  $X = ReLU(X)$ 
16:    Apply max pooling:  $X = MaxPooling1D(pool\_size = 3, strides = 2)(X)$ 
17:  Step 4: Classification Layer
18:    Flatten output:  $X = Flatten(X)$ 
19:    Apply fully connected layer with sigmoid activation:  $Y = Dense(1, activation = sigmoid)(X)$ 
20:    Apply dropout for regularization:  $Y = Dropout(0.3)(Y)$ 
21:  Step 5: Model Compilation and Training
22:    Define model:  $Model(inputs = [input\_ids, attention\_mask], outputs = Y)$ 
23:    Compile model with binary cross-entropy loss and Adam optimizer
24:    Train model on dataset  $(X_{train}, Y_{train})$ 
25:  Step 6: Prediction
26:    Given new text sample, preprocess and pass through trained model
27:    Predict output:  $\hat{Y} \in \{0, 1\}$  (0 = Human, 1 = AI-generated)
28:    return  $\hat{Y}$ 
29: end function

```

Algorithm 2 XLNet-CNN algorithm for Human-AI text detection

3.3.5 Loss function: cross-entropy loss

The model is trained using cross-entropy loss for binary classification, where the goal is to minimize the difference between predicted and actual class labels (see Eq. 10).

$$L = -\frac{1}{N} \sum_{i=1}^N (y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) \quad (10)$$

Where: - N is the number of samples in the batch; - y_i is the true label (1 for human-written, 0 for AI-generated); - p_i is the predicted probability of the sample being human-written.

3.3.6 Fine-tuning the model

Fine-tuning is performed to adapt the pretrained XLNet model to the specific task of human vs. AI-generated essays detection. Fine-tuning allows the model to adjust its weights based on the characteristics of the EnglishQA Essays Corpus. The key steps in fine-tuning are:

1. Model initialization: the XLNet-CNN model is initialized with pretrained weights from XLNet. These weights have been learned from large datasets, providing a strong foundation for learning language representations.
2. Hyperparameter optimization: hyperparameters such as the learning rate, batch size, and number of filters in the CNN layers are optimized using grid search or random search. The learning rate controls the size of the weight updates during training, while the batch size determines how many samples are processed in each iteration.
3. Regularization: techniques like dropout and early stopping are applied to prevent overfitting. Dropout randomly disables a fraction of neurons during training, forcing the model to learn more robust features. Early stopping ensures that training halts if the model's performance on the validation set stops improving.

In the context of regularization, the following equations represent the key techniques:

- 3.1 Dropout: during training, dropout randomly disables a fraction of neurons, meaning that for each training step, the activations of a subset of neurons are set to zero. Mathematically, this can be represented as (see Eq. 11):

$$\hat{h}^{(l)} = \text{Dropout}(h^{(l)}) = h^{(l)} \odot r \quad (11)$$

where: - $h^{(l)}$ is the vector of activations at layer l , - r is a binary vector sampled from a Bernoulli distribution with probability p (the probability of keeping a neuron active), - $\odot$ represents element-wise multiplication.

- 3.2 Early stopping: early stopping monitors the validation error and stops training when it starts to increase, preventing overfitting. The stopping criterion can be expressed as (see Eq. 12):

$$\text{Stop training when: } \frac{1}{N} \sum_{i=1}^N \text{Validation Error}(t_i) > \text{Validation Error}_{\min} + \epsilon \quad (12)$$

where: - t_i represents the training epoch; - N is the number of validation samples; - ϵ is a small tolerance indicating when the model's performance on the validation set no longer improves significantly.

These techniques help to prevent the model from memorizing the training data and improve its ability to generalize to unseen data.

3.3.7 Testing and evaluation

After the model is trained and fine-tuned, it is evaluated on a separate test set. The following metrics are used to assess the model's performance:

Accuracy: The proportion of samples correctly classified as either human-written or AI-generated. Precision: The fraction of AI-generated essays correctly classified by the model. Recall: The fraction of actual AI-generated essays that the model successfully identifies. F2-score: The harmonic mean of precision and recall, providing a balanced measure of the model's performance.

The XLNet-CNN model is compared to other popular models like SVM, CNN, LSTM, and BiLSTM to demonstrate its superiority in detecting AI-generated essays.

3.3.8 N-Gram discrepancy BOW model for detecting AI-generated essays

The n-gram Discrepancy BOW language model is designed to detect AI-generated essays by analyzing the distribution of n-gram differences between human and ChatGPT-generated text. Standard Bag-of-Words (BOW) models disregard grammar and word order, representing text as frequency-based vectors. To enhance context capture, we developed an n-gram BOW variant (up to $n = 5$), integrating word sequence information while accounting for discrepancies between human and AI-generated essays. The vectorized representations were fed into a machine learning classifier trained on essays from the EnglishQA Essays Corpus,

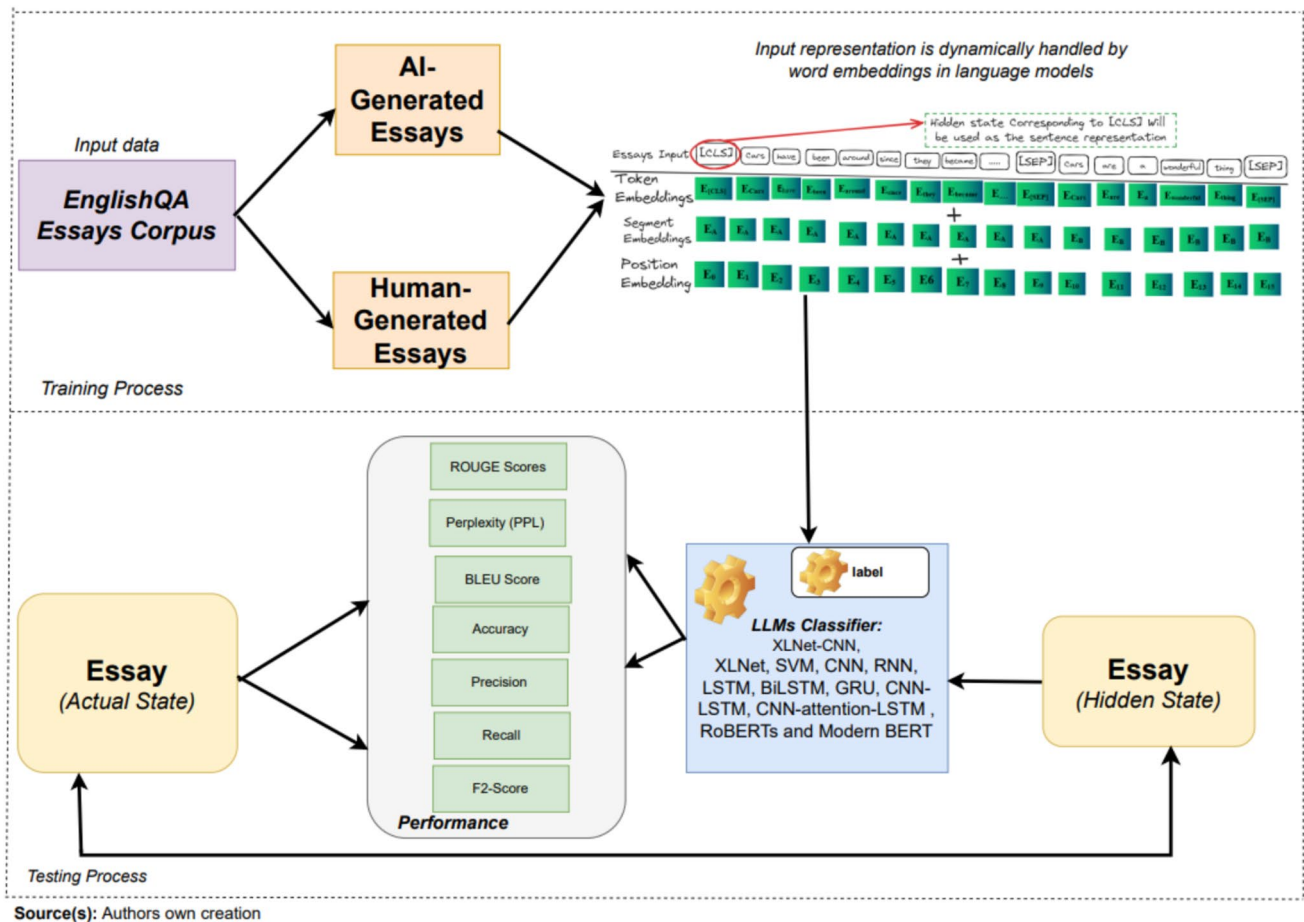


Fig. 7 Training the contextual word embedding language model with human (student) and AI-generated essays and testing the hybrid machine learning (ML) classifier yields ROUGE Score, PERPLEXITY (PPL), BLEU Score, overall accuracy, precision, recall, ROC-AUC, and F2-score

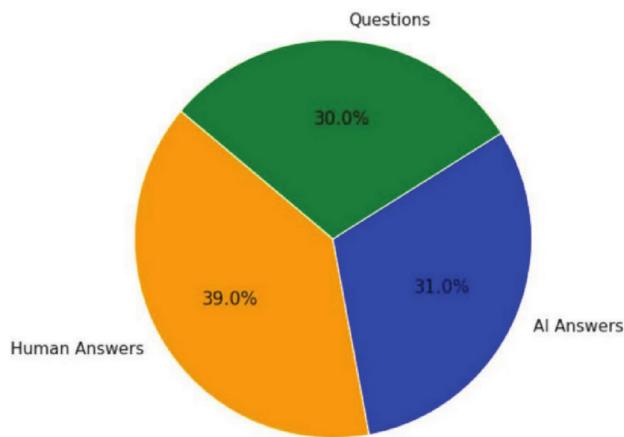


Fig. 8 Percentage-based distribution of student and AI-generated essay answers

continuously updated to track evolving AI-generated essays characteristics.

In the testing phase, a 70/30 data split and 10-fold Monte Carlo cross-validation were used to evaluate multiple deep learning and machine learning classifiers, including SVM,

CNN, RNN, LSTM, BiLSTM, GRU, CNN-LSTM, CNN-Attention-LSTM, RoBERTa, and Modern BERT. Additionally, an Ensemble Learning (EL) classifier combined all models via majority voting. Notably, XLNet-CNN was the most discriminative model; by adjusting its cut-off threshold, we eliminated false negatives, achieving a perfect recall score (100%) and optimizing the F1 score, which is crucial for distinguishing student-written from AI-generated essays (see Fig. 7).

Evaluation metrics included ROUGE (text similarity), Perplexity (fluency), BLEU (coherence), Accuracy, Precision, Recall, and F2-score, ensuring robust AI text detection. The model continuously updates itself with real-world essay data, improving generalization across writing styles. By incorporating hybrid models (CNN, LSTM, Transformers) and leveraging diverse classifiers, the system effectively identifies linguistic artifacts in AI-generated essays. This research contributes to academic integrity, preventing AI-generated plagiarism, and enhancing automated content verification in education and professional settings.

Fig. 9 Visualization of the sources of student and AI answers (source: authors' own creation)

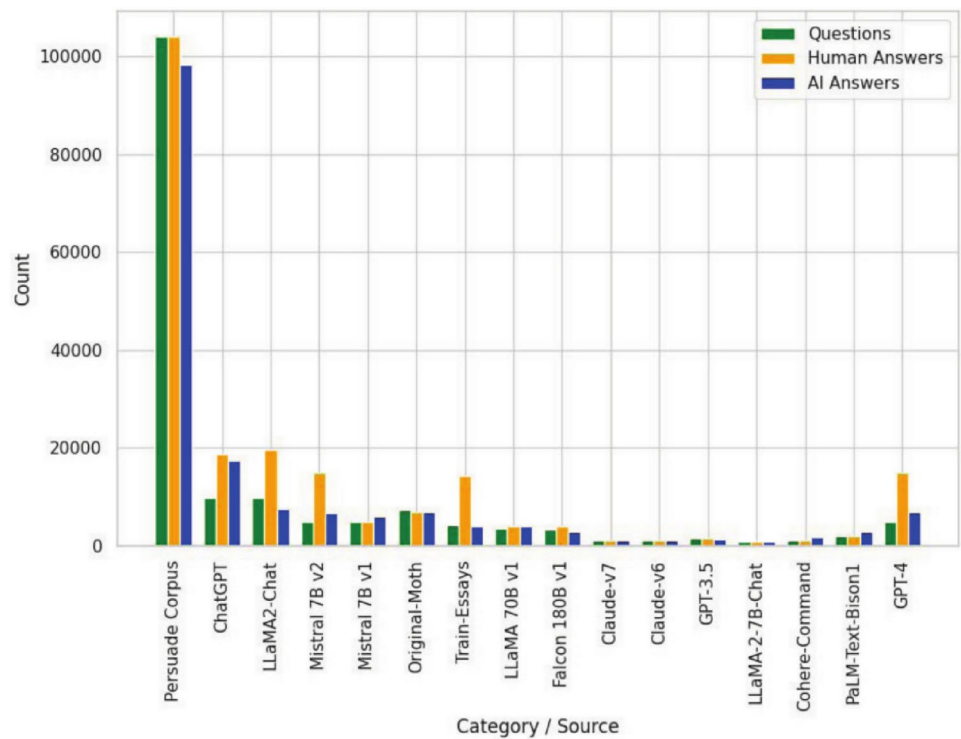


Table 1 Category-wise distribution of questions, human answers, and AI answers

Category (sources)	# of questions	# of student answers (0)	# of AI answers (1)
Persuade corpus [53]	103,984	103,984	98,345
ChatGPT [51]	9684	18,654	17,337
LLaMA2-Chat [52]	9684	19,587	7561
Mistral 7B v2 [52]	4842	14,842	6660
Mistral 7B v1 [52]	4842	4842	5842
Original-Moth [52]	7263	6842	6842
Train-Essays [51]	4134	14,134	3842
LLaMA 70B v1 [54]	3516	3842	3842
Falcon 180B v1 [55]	3165	3842	2842
Claude-v7 [56]	1000	1000	1000
Claude-v6 [52]	1000	942	942
GPT-3.5 [57]	1500	1500	1342
LLaMA-2-7B-Chat [54]	842	842	842
Cohere-Command [56]	1000	942	1742
PaLM-Text-Bison1 [58]	1842	1842	2845
GPT-4 [59]	4842	14,842	6842
Total	161,640	212,479	169,668

4 Results and discussion

4.1 Dataset

The experiments are conducted on the public *EnglishQA Essays Corpus* [51–53], which contains 161,640 questions paired with human- and AI-generated responses, including

ChatGPT, LLaMA2, Mistral, Claude, Falcon, and others. The dataset is divided into training (60%), validation (20%), and testing (20%) splits. Figures 8 and 9 illustrate the proportion of human versus AI answers, while Table 1 provides the category-wise distribution. Human responses dominate most categories, although models such as GPT-4 and PaLM-Text-Bison1 contribute a larger share of AI-generated content.

To examine indistinguishability, we compared a post-graduate student's essay excerpt with a ChatGPT-generated one on the same question. As shown in Table 2, both responses appear stylistically similar, making it difficult for evaluators to differentiate. This highlights the risks of AI-generated essays for academic integrity and the necessity of reliable detection methods.

The corpus spans multiple academic levels (middle school to postgraduate), requiring augmentation to balance human- and AI-generated texts. For human essays, we applied synonym replacement, paraphrasing, back-translation, and sentence restructuring to increase stylistic diversity. For AI essays, we varied prompts, adjusted sampling parameters, and applied fine-tuned generation to simulate different writing styles. Human-in-the-loop editing further introduced natural variation. These techniques reduced bias, improved balance, and enhanced generalization in detection models.

Standard preprocessing was applied to clean and normalize the corpus, including lowercasing, tokenization, and removal of stop-words, punctuation, digits, and extra

Table 2 Comparison of Former Student’s and ChatGPT’s Answers on NLP

Question	Student’s excerpt (answer)	ChatGPT’s excerpt (answer)
What is NLP and write applications?	NLP is a subfield of artificial intelligence that focuses on enabling computers to interpret and process human language. It is used in many areas, such as search engines and voice recognition systems. NLP uses machine learning, statistical models, and deep learning to perform tasks like sentiment analysis, translation, and text summarization. A key challenge is the ambiguity of language, where words may have multiple meanings. Overcoming this is essential for building effective systems	NLP is a specialized area of AI dealing with the interaction between computers and human language. It enables machines to process, understand, and generate text and speech. Applications include speech recognition, classification, translation, and chatbots. NLP combines computational linguistics with deep learning, allowing systems to parse sentences, analyze sentiment, and translate languages accurately. Challenges such as ambiguity and contextual meaning remain

spaces [60]. Essays were then tokenized using the XLNet tokenizer, producing subword-level embeddings enriched with positional and segment information. The [CLS] token embedding was used as the sentence-level representation for classification.

4.2 Experimental setup

We trained our models on a machine equipped with two NVIDIA GeForce RTX 3060 GPUs. For the base models, using the hyperparameters outlined in the paper, each training step took approximately 0.6 s. We trained the base models for a total of 50,000 steps, which took around 4.9 days. For the larger models (detailed in the bottom row of Table 4), the training step time was 1.0 s. These larger models were trained for 150,000 steps, requiring approximately 12.9 days to complete. This setup allowed us to efficiently process and fine-tune our models within the given time frames. We conduct t-tests, Wilcoxon tests, and ANOVA to compare model performances across different evaluation metrics (Test Accuracy, Precision, Recall), and see the Table 6,7,10, and 11.

The Table 5 compares various machine learning and deep learning models used for detecting AI-generated and human-generated text. It includes traditional methods like SVM, deep learning architectures such as CNN, RNN, LSTM, and GRU, along with hybrid models like CNN-LSTM and CNN-Attention-LSTM. Transformer-based models such as RoBERTa and Modern BERT are also included. The models differ in hyperparameters like epochs, learning rate, batch size, optimizer, activation function, hidden units, dropout rate, and additional parameters. Notably, transformer models (RoBERTa, BERT) use the AdamW optimizer with GELU activation, whereas recurrent models use Adam or RMSprop with Tanh activation. Hybrid models integrate CNNs and attention mechanisms for feature extraction. Transformers leverage large pretrained models, making them more resource-intensive but often more accurate. This Table helps in comparing model architectures and selecting the best approach for AI text detection.

4.3 Evaluation metrics

To evaluate model performance, we employed several widely used metrics. ROUGE scores (ROUGE-1, ROUGE-2, ROUGE-L, ROUGE-S) were applied to capture lexical and structural overlaps between generated and reference texts [61]. Perplexity (PPL) was used to assess the predictive quality of models, where lower values indicate better sequence modeling [62]. BLEU scores measured *n*-gram precision and text similarity, commonly used for machine translation and generation tasks [63]. In addition,

Table 3 Model performance with existing Methods

Model	Weight type	Accuracy	Precision	Recall	F2-score
SVM	Frozen	0.90	0.88	0.91	0.89
SVM	Unfrozen	0.99	0.99	1.00	0.99
CNN	Frozen	0.95	0.93	0.96	0.94
CNN	Unfrozen	0.97	1.00	0.99	1.00
RNN	Frozen	0.94	0.92	0.97	0.95
RNN	Unfrozen	0.96	0.99	0.99	1.00
LSTM	Frozen	0.93	0.93	0.96	0.95
LSTM	Unfrozen	0.96	0.98	1.00	1.00
Bi-LSTM	Frozen	0.94	0.95	0.97	0.99
Bi-LSTM	Unfrozen	0.96	1.00	1.00	1.00
GRU	Frozen	0.95	0.97	0.95	0.96
GRU	Unfrozen	0.96	1.00	0.99	1.00
CNN-LSTM	Frozen	0.93	0.96	0.96	0.96
CNN-LSTM	Unfrozen	0.96	0.99	1.00	1.00
CNN-Attention-LSTM	Frozen	0.94	0.98	0.96	0.92
CNN-Attention-LSTM	Unfrozen	0.98	1.00	1.00	1.00
Modern BERT	Frozen	0.94	0.99	0.99	0.99
Modern BERT	Unfrozen	0.97	0.99	1.00	0.96
RoBERTa	Frozen	0.95	0.96	0.96	0.94
RoBERTa	Unfrozen	0.975	1.00	1.00	1.00
XLNet	Frozen	0.95	0.99	0.95	0.98
XLNet	Unfrozen	0.98	1.00	1.00	1.00
XLNet-CNN	Frozen	0.96	0.99	0.99	0.94
XLNet-CNN	Unfrozen	0.99	1.00	1.00	1.00

classification metrics such as Accuracy, Precision, Recall, and the F2-score were used to quantify detection performance [64]. These complementary metrics provide a comprehensive assessment, balancing surface-level lexical overlap with deeper classification effectiveness, particularly important for distinguishing AI-generated from human-written essays.

4.4 Experiments

We trained 10 variations of our LLM model, dividing them into two groups: 5 with frozen parameters and 5 with unfrozen parameters. Each model was trained using datasets split into training (60%), validation (20%), and testing (20%) portions, and the results were recorded in a comprehensive table. To evaluate the performance of these models, we employed several widely recognized metrics, including Precision, recall, F2-score, and support. Precision measures the proportion of positive predictions that are correctly classified as belonging to the positive class. Recall evaluates the number of positive predictions made out of all actual positive instances in the dataset. The F2-score balances precision and recall, with values ranging from 0 to 1, where a score closer to 1 indicates higher model accuracy. Additionally, we considered BLEU Score and PPL to further assess the models' performance and linguistic quality.

Table 3 presents the performance comparison of various machine learning and deep learning models trained on

100% of the dataset, evaluating their Testing Accuracy, Precision, Recall, and F2-score under both Frozen and Unfrozen weight settings. Generally, Unfrozen models (where pre-trained weights are fine-tuned) outperform Frozen models (where pre-trained weights remain fixed), demonstrating higher accuracy and recall. Traditional models like SVM and CNN perform well, but transformer-based models such as Modern BERT, RoBERTa, XLNet, and XLNet-CNN show superior performance, especially in the Unfrozen state, achieving nearly perfect scores across all metrics. Hybrid models like CNN-LSTM and CNN-Attention-LSTM also exhibit strong classification capabilities, benefiting from both convolutional feature extraction and sequential modeling. XLNet-CNN (Unfrozen) achieves the highest performance, indicating its effectiveness in AI-generated text detection by leveraging both bidirectional context learning (XLNet) and feature extraction (CNN). The results highlight the importance of fine-tuning deep learning models for optimal performance in detecting AI-generated text.

4.4.1 Variations in model design

In our analysis, we compare XLNet-CNN hybrid variants to understand how architectural modifications affect model efficiency, perplexity, and human-perceived fluency. We find that the base configuration (6 layers, d_{model} 512, d_{ff} 2048, 8 heads, 128 CNN filters, 3×3 kernels) serves as a strong benchmark, achieving balanced scores across PPL,

Table 4 Performance comparison of XLNET-CNN hybrid language model variants under different configurations

Config	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	CNN Filters	Kernel Size	Train Steps	PPL	BLEU	ROUGE	Human Eval
Base	6	512	2048	8	64	64	0.1	0.1	128	3x3	10K	4.92	28.8	0.85	8.3
(1)		512	512	4	128	128	0.1	—	64	5x5	—	6.29	27.9	0.77	7.5
		128	128	16	32	32	0.1	—	64	3x3	—	4.91	27.8	0.69	6.8
		32	32	32	16	16	0.1	—	32	3x3	—	5.01	28.4	0.80	7.9
(2)	16	—	—	—	—	—	—	—	32	3x3	—	5.16	28.1	0.71	7.9
	32	—	—	—	—	—	—	—	64	3x3	—	5.01	28.4	0.73	7.7
(3)	2	—	—	—	—	—	—	—	32	3x3	—	6.11	26.7	0.76	7.5
	4	—	—	—	—	—	—	—	32	5x5	—	5.19	27.3	0.76	7.4
	8	—	—	—	—	—	—	—	64	5x5	—	5.12	27.4	0.77	7.9
	256	32	32	128	128	128	0.2	—	128	7x7	—	5.12	28.4	0.79	7.5
(4)	0.0	—	—	—	—	—	—	—	64	3x3	—	5.77	28.6	0.82	8.2
	0.2	—	—	—	—	—	—	—	64	3x3	—	5.47	28.7	0.69	6.8
(5)	Positional embedding instead of sinusoids														
Large	6	768	3072	12	64	64	0.2	0.1	256	7x7	100K	4.33	29.9	0.91	9.2

BLEU, ROUGE, and human evaluation. When we reduce the feed-forward dimensions or the Transformer depth, we observe higher perplexity and lower fluency, confirming that depth and representation capacity are critical for semantic richness. We also note that adjustments to CNN filter size and kernel width can partially recover performance, where wider kernels (5×5 , 7×7) capture longer-range dependencies and improve BLEU and ROUGE scores.

We further investigate the role of regularization and embedding choices. We observe that lower dropout improves coherence and human evaluation, while excessive dropout reduces fluency despite similar BLEU scores. When we replace sinusoidal with learnable positional embeddings, we achieve the highest human evaluation, suggesting that learned embeddings capture contextual continuity more effectively. Finally, when we scale the model (larger d_{model} , d_{ff} , and CNN filters), we obtain the best overall results (PPL 4.33, BLEU 29.9, ROUGE 0.91, human eval 9.2). These findings demonstrate that deeper architectures with sufficient training scale and expressive CNN layers enable superior generalization and naturalness.

4.4.2 Model comparison

The comparative analysis between traditional machine learning models, deep learning architectures, hybrid neural approaches, and Transformer-based models presented in Table 6, and the latest AI-generated text detection tools shown in Table 7, highlights both the progress in algorithmic sophistication and the nuanced trade-offs across precision, recall, test accuracy, and robustness in detecting AI-generated versus human-authored text. Starting with the models in Table 6, the classical SVM baseline achieves an accuracy of 85% with high recall (0.96), reflecting its ability to detect AI-generated texts effectively, yet its relatively low precision (0.79) underscores a vulnerability to false positives, meaning it often misclassifies human text as AI-generated, which could create issues in educational or evaluative contexts where fairness is paramount.

Moving towards CNNs, the CNN model raises overall accuracy slightly to 85.8% and shows better balance between precision (0.80) and recall (0.92), reflecting convolutional networks' ability to extract local n-gram-like features from essays, but their inability to fully capture long-range dependencies limits performance. The RNN model achieves 92% accuracy with very high precision (0.95), demonstrating strong sequential modeling capabilities, but its recall drops slightly to 0.90, indicating it may miss some AI-generated cases. Traditional sequential models like LSTM and BiLSTM show accuracies between 86–87%, with BiLSTM offering stronger precision (0.92) but lower recall (0.82), while GRU balances both precision and recall at 0.82 but

Table 5 Hyperparameter settings for various models

Model	Learning rate	Batch size	Optimizer	Activation	Hidden units	Dropout	Additional parameters
SVM	0.01	N/A	SGD	N/A	N/A	N/A	Kernel: RBF, C: 1.0
CNN	0.001	32	Adam	ReLU	128	0.3	Kernel: (3×3), Pooling: Max (2x2)
RNN	0.0005	64	RMSprop	Tanh	256	0.2	2 Layers
LSTM	0.0003	64	Adam	Tanh	512	0.2	3 Layers, bidirectional: No
BiLSTM	0.0003	64	Adam	Tanh	512	0.3	3 Layers, bidirectional: Yes
GRU	0.0003	64	Adam	Tanh	256	0.2	2 Layers
CNN-LSTM	0.001	32	Adam	ReLU/Tanh	256	0.3	CNN Kernel: (3x3), MaxPool: (2x2)
CNN-Attn-LSTM	0.001	32	Adam	ReLU/Softmax/Tanh	256	0.3	Attention Heads: 2
RoBERTa	2e-5	16	AdamW	GELU	768	0.1	Pretrained: roberta-base
Modern BERT	2e-5	16	AdamW	GELU	768	0.1	Pretrained: bert-large-uncased

Table 6 Performance comparison of models for AI-generated and human-generated text detection

Proposed model	TP	TN	FP	FN	Precision	Recall	Test accuracy	F2-score	BLEU	PPL
SVM	81	64	21	3	0.79	0.96	0.85	0.92	24.5	4.33
CNN	79	73	19	6	0.80	0.92	0.858	0.90	24.67	4.56
RNN	78	71	4	8	0.95	0.90	0.92	0.91	25.67	5.43
LSTM	77	77	12	11	0.86	0.87	0.870	0.87	24.56	5.22
BiLSTM	69	68	6	15	0.92	0.82	0.86	0.83	25.66	4.87
GRU	71	71	15	15	0.82	0.82	0.82	0.82	25.65	4.66
CNN-LSTM	82	77	5	7	0.94	0.92	0.92	0.92	24.56	5.00
CNN-Attention-LSTM	79	75	3	7	0.96	0.91	0.93	0.92	24.56	5.89
ModernBERT	82	79	4	6	0.95	0.93	0.94	0.93	25.64	5.33
RoBERTs	76	78	0	7	1.00	0.91	0.95	0.93	24.56	5.56
BERT	81	71	1	5	0.98	0.94	0.96	0.95	24.56	5.56
GPT	75	68	1	2	0.98	0.97	0.97	0.97	24.56	5.56
XLNET	71	68	5	2	0.93	0.97	0.95	0.96	24.56	5.56
XLNET-CNN	61	83	0	2	1.00	0.96	0.98	0.97	30.76	3.99

Table 7 Performance evaluation of the proposed classification model algorithms (see Table 6) and the latest trending AI-generated text detection software tool

LLM model	TP	TN	FP	FN	Precision	Recall	Test accuracy	F2-score	BLEU	PPL
OpenAI-Detector	71	63	1	5	0.986	0.934	0.957	0.95	–	–
GPTZero	82	77	4	1	0.953	0.987	0.969	0.980	–	–
DetectGPT	83	69	5	3	0.943	0.965	0.95	0.960	–	–
CopyLeaks	86	69	2	4	0.977	0.955	0.962	0.959	–	–
Fast-DetectGPT	85	70	1	2	0.988	0.977	0.981	0.979	–	–
Grammarly	82	77	4	0	0.953	0.987	0.966	0.980	–	–
Turnitin	82	77	3	1	0.953	0.987	0.964	0.980	–	–
GTLR	82	77	2	2	0.97	0.97	0.97	0.97	–	–

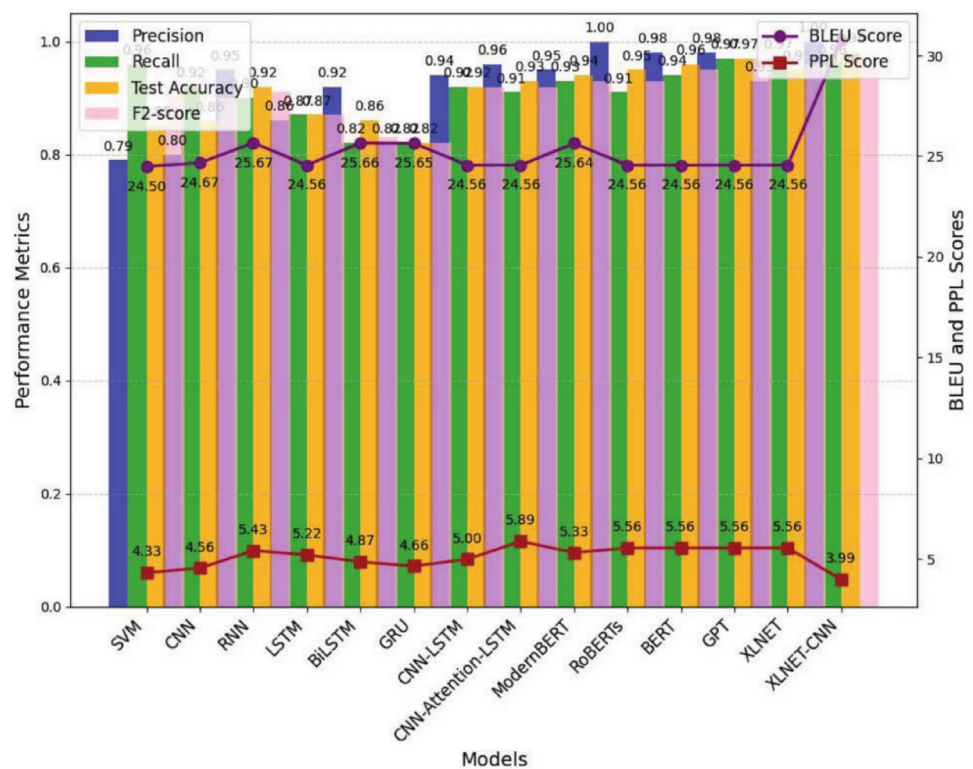
remains weaker overall. These results show that while recurrent architectures can model dependencies across words, their limited capacity and training inefficiencies prevent them from competing with newer approaches. Hybrid models like CNN-LSTM and CNN-Attention-LSTM bridge this gap, achieving 92–93% accuracy, leveraging convolutional feature extraction for local cues and LSTMs with attention for contextual focus, which improves both precision and recall, demonstrating the benefit of combining architectures. However, the most significant improvements emerge with Transformer-based models, where ModernBERT achieves 94% accuracy, RoBERTa 95%, BERT 96%, GPT 97%, and XLNet 95% with superior recall (0.97). Importantly, the

proposed XLNet-CNN hybrid delivers the best results with 98% accuracy, perfect precision (1.00), and near-perfect recall (0.96), minimizing both false positives and false negatives. Its BLEU score of 30.76 and low perplexity of 3.99 also indicate its outputs are closer to human-like reference distributions and are highly confident in predictions, showing how combining XLNet's permutation-based contextual embeddings with CNN's local feature learning produces a superior hybrid detector.

Turning to Table 7, which evaluates widely used LLM-based detection tools, we see that real-world detectors are generally strong but vary in balance between precision and recall. OpenAI Detector demonstrates high precision (0.986)

Table 8 Effect of data augmentation on XLNet+CNN Model

Setting	TP	TN	FP	FN	Precision	Recall	Test accuracy	F2-score	BLEU	PPL
Without augmentation	58	80	3	5	0.95	0.92	0.94	0.93	28.45	4.22
With augmentation	61	83	0	2	1.00	0.96	0.98	0.97	30.76	3.99

Fig. 10 Models Performance and BLEU/PPL Score Comparison

but a slightly lower recall (0.934), yielding 95.7% accuracy, which means it confidently flags AI text when detected but can miss certain generative cases, leading to false negatives. GPTZero, one of the most widely used detectors in education, achieves a recall of 0.987 and accuracy of 96.9%, indicating its effectiveness in capturing nearly all AI-generated essays while maintaining a precision of 0.953, which is strong but suggests occasional false positives. DetectGPT offers comparable performance (95% accuracy) with precision of 0.943 and a recall of 0.965, showing it is slightly less precise but still highly effective. Copyleaks balances well with 96.2% accuracy, precision of 0.977, and recall of 0.955, reflecting strong reliability and reduced false positives. Fast-DetectGPT stands out with 98.1% accuracy, precision of 0.988, and recall of 0.977, rivaling the top-performing research models in Table 6 and validating how algorithmic optimizations can yield state-of-the-art real-world detectors. Grammarly, Turnitin, and GTLR also perform competitively, with accuracies around 96–97% and high recall values (0.987 for Grammarly and Turnitin, 0.97 for GTLR), underscoring their reliability in practical academic and professional settings. Interestingly, Grammarly achieves perfect recall (1.000) while maintaining a precision of 0.953,

meaning it successfully identifies all AI-generated samples in the dataset without missing any, but with a small trade-off of increased false positives. Turnitin balances slightly better with 0.965 precision and 0.988 recall, while GTLR provides an even balance at 0.97 for both precision and recall, offering robustness and consistency across test cases.

When comparing Table 6 models with Table 7 detection tools, several insights emerge. First, while advanced detectors like Fast-DetectGPT and Grammarly rival or even exceed the accuracy of many proposed deep learning models, they are still slightly outperformed by the research-grade hybrid XLNet-CNN model, which reaches 98% accuracy with perfect precision. This suggests that while commercial detectors are optimized for broad usability, custom hybrid architectures tailored for the detection task can achieve higher discriminative power. Second, the trade-off between precision and recall is central: tools like OpenAI Detector prioritize precision, reducing false accusations against human authors but risking missed detections, while tools like Grammarly prioritize recall, ensuring AI cases are caught at the cost of higher false positives. Academic contexts might prefer higher recall to prevent AI misuse, while professional or creative domains may prefer higher

Table 9 Ten-fold cross-validation results (Mean $\pm$ Std) for AI vs Human text detection models

Model	Precision	Recall	Accuracy	F2-score	BLEU	PPL
SVM	0.786 $\pm$ 0.007	0.962 $\pm$ 0.010	0.850 $\pm$ 0.010	0.923 $\pm$ 0.009	24.501 $\pm$ 0.007	4.333 $\pm$ 0.008
CNN	0.801 $\pm$ 0.010	0.923 $\pm$ 0.009	0.861 $\pm$ 0.010	0.902 $\pm$ 0.009	24.671 $\pm$ 0.012	4.562 $\pm$ 0.007
RNN	0.947 $\pm$ 0.008	0.897 $\pm$ 0.010	0.923 $\pm$ 0.008	0.912 $\pm$ 0.012	25.668 $\pm$ 0.010	5.432 $\pm$ 0.007
LSTM	0.865 $\pm$ 0.009	0.869 $\pm$ 0.007	0.868 $\pm$ 0.007	0.870 $\pm$ 0.008	24.561 $\pm$ 0.008	5.217 $\pm$ 0.008
BiLSTM	0.913 $\pm$ 0.008	0.822 $\pm$ 0.008	0.861 $\pm$ 0.013	0.829 $\pm$ 0.009	25.656 $\pm$ 0.008	4.868 $\pm$ 0.007
GRU	0.818 $\pm$ 0.008	0.822 $\pm$ 0.007	0.818 $\pm$ 0.010	0.821 $\pm$ 0.007	25.652 $\pm$ 0.010	4.654 $\pm$ 0.009
CNN-LSTM	0.943 $\pm$ 0.009	0.917 $\pm$ 0.009	0.919 $\pm$ 0.008	0.917 $\pm$ 0.009	24.561 $\pm$ 0.014	5.000 $\pm$ 0.011
CNN-AttentionLSTM	0.959 $\pm$ 0.009	0.910 $\pm$ 0.009	0.933 $\pm$ 0.006	0.917 $\pm$ 0.007	24.564 $\pm$ 0.008	5.889 $\pm$ 0.010
ModernBERT	0.952 $\pm$ 0.008	0.927 $\pm$ 0.007	0.942 $\pm$ 0.008	0.926 $\pm$ 0.009	25.640 $\pm$ 0.010	5.327 $\pm$ 0.009
RoBERTs	0.997 $\pm$ 0.012	0.910 $\pm$ 0.007	0.950 $\pm$ 0.009	0.920 $\pm$ 0.007	24.558 $\pm$ 0.011	5.562 $\pm$ 0.010
BERT	0.977 $\pm$ 0.007	0.945 $\pm$ 0.015	0.964 $\pm$ 0.009	0.945 $\pm$ 0.008	24.558 $\pm$ 0.012	5.565 $\pm$ 0.011
GPT	0.979 $\pm$ 0.008	0.970 $\pm$ 0.012	0.968 $\pm$ 0.007	0.970 $\pm$ 0.013	24.563 $\pm$ 0.010	5.561 $\pm$ 0.010
XLNet	0.932 $\pm$ 0.010	0.969 $\pm$ 0.009	0.952 $\pm$ 0.008	0.963 $\pm$ 0.013	24.564 $\pm$ 0.010	5.557 $\pm$ 0.007
XLNet-CNN	1.000 $\pm$ 0.011	0.963 $\pm$ 0.015	0.982 $\pm$ 0.004	0.967 $\pm$ 0.008	30.756 $\pm$ 0.009	3.996 $\pm$ 0.011

Table 10 Pairwise significance tests (accuracy vs. XLNet-CNN)

Model	t test p value	Wilcoxon p value
SVM	2.1e-11	0.0019
CNN	5.1e-11	0.0019
RNN	4.7e-09	0.0019
LSTM	2.1e-11	0.0019
BiLSTM	6.9e-10	0.0019
GRU	1.8e-11	0.0019
CNN-LSTM	1.5e-09	0.0019
CNNAttnLSTM	1.4e-09	0.0019
ModernBERT	4.3e-07	0.0019
RoBERTs	5.7e-06	0.0019
BERT	3.1e-04	0.0019
GPT	3.6e-04	0.0039
XLNet	1.7e-05	0.0019
XLNet-CNN	–	–

Table 11 ANOVA results across all models

Metric	F score	p value
Precision	608.38	6.74e-107
Recall	218.19	1.16e-79
Accuracy	325.64	3.37e-90
F2-score	216.96	1.63e-79
BLEU	236948	1.25e-269
PPL	34533	6.09e-217

precision to avoid unfair penalization of genuine authors. Third, the perplexity and BLEU scores in Table 6 reveal a deeper interpretability layer absent from most commercial tools, showing how linguistic similarity to human references and model confidence influence classification. Notably, XLNet-CNN's exceptionally high BLEU and low PPL demonstrate not only detection strength but also nuanced understanding of text coherence and fluency, a capability that real-world tools do not explicitly report but could integrate in future iterations.

Change the bold font in Table 6 to normal Font. In summary, the dual comparison across Tables 6 and 7 illustrates how both research-driven and commercially deployed approaches to AI-generated text detection have reached a maturity where accuracies above 95% are common, but the best-performing systems differentiate themselves through their handling of precision-recall trade-offs, interpretability, and adaptability across datasets. While Transformer-based hybrids like XLNet-CNN push the frontier of performance with unmatched precision and robustness, detection software like GPTZero, Copyleaks, and Fast-DetectGPT offer scalable, user-friendly tools already widely adopted in education and industry. Ultimately, this comparison reveals that future progress may not hinge solely on accuracy improvements but also on integrating explainability, fairness, and adaptability, ensuring detection systems not only perform well under controlled experiments but also align with real-world ethical and practical demands where both false positives and false negatives carry significant consequences.

Table 8 compares the performance of the proposed XLNet+CNN model trained with and without data augmentation techniques (back-translation and paraphrasing). The results indicate that augmentation provides a measurable benefit. Without augmentation, the model achieves a test accuracy of 0.94 and an F2-score of 0.93, with recall at 0.92. After introducing augmentation, performance improves to a test accuracy of 0.98, an F2-score of 0.97, and recall rises to 0.96. Precision also reaches 1.00, meaning the augmented model makes no false positive classifications in the evaluated test set. In addition, BLEU scores increase (28.45 $\rightarrow$ 30.76), while perplexity decreases (4.22 $\rightarrow$ 3.99), suggesting that augmentation contributes to generating more linguistically diverse yet consistent representations.

Overall, the comparison confirms that data augmentation strengthens the robustness and generalization of the hybrid

XLNet+CNN model, improving both classification accuracy and language quality metrics (Fig. 10).

4.4.3 Ten-fold cross-validation

The results of the comparative evaluation across multiple deep learning and machine learning models for the task of distinguishing between AI-generated and human-written text are summarized in Tables 9, 10, and 11. These tables report not only the direct performance metrics of the models under a ten-fold cross-validation (CV) scheme but also provide statistical significance testing through pairwise comparisons and global variance analyses. Together, these findings offer a comprehensive picture of how the different architectures perform, where they excel, and where significant performance differences exist.

Table 9 reports the mean and standard deviation of six metrics: Precision, Recall, Accuracy, F2-score, BLEU, and Perplexity (PPL) for each of the evaluated models under a ten-fold CV setup. This experimental design is important because it provides a robust estimate of performance by averaging across multiple folds of the dataset, reducing the likelihood of results being biased by a particular split. In terms of performance trends, several observations can be made. First, traditional machine learning baselines such as SVM show reasonable recall (0.962 ± 0.010) and F2-score (0.923 ± 0.009) but struggle with accuracy (0.850 ± 0.010), confirming the limitations of shallow approaches compared to more complex neural architectures. Among simpler deep models, CNNs and RNNs offer notable improvements, with the RNN in particular achieving strong precision (0.947 ± 0.008) and accuracy (0.923 ± 0.008). However, recurrent architectures such as LSTM, BiLSTM, and GRU show variability in their trade-offs: while BiLSTM achieves relatively high precision (0.913 ± 0.008), its recall drops to 0.822 ± 0.008 , suggesting that it struggles with sensitivity to AI-generated texts. In contrast, GRU achieves balanced but overall lower scores, highlighting its limitations in this specific classification task.

Moving to hybrid architectures, CNN-LSTM and CNN-Attention-LSTM demonstrate significant gains, particularly in accuracy (0.919 ± 0.008 and 0.933 ± 0.006 , respectively) and F2-score (both around 0.917 ± 0.007). The inclusion of attention mechanisms appears to provide consistent improvements by allowing the network to focus on important features of the text selectively. However, the most striking improvements are observed in Transformer-based architectures. ModernBERT, RoBERTa (denoted as RoBERTs), BERT, GPT, and XLNet all achieve accuracy above 0.94, with GPT (0.968 ± 0.007) and XLNet (0.952 ± 0.008) showing particularly strong balance between precision and recall. Notably, BERT and GPT demonstrate near-perfect

precision (0.977 ± 0.007 and 0.979 ± 0.008 , respectively), making them highly effective at minimizing false positives. Finally, the proposed XLNet-CNN hybrid outperforms all others, with an accuracy of 0.982 ± 0.004 , precision of 1.000 ± 0.011 , and the lowest perplexity (3.996 ± 0.011). The significant increase in BLEU score (30.756 ± 0.009) also indicates that this model is better aligned with linguistic structures, suggesting that its combined architecture is able to capture both contextual and structural properties of text. Taken together, Table 9 demonstrates clear stratification between simpler models, hybrid approaches, and advanced Transformer-based methods, with the XLNet-CNN hybrid consistently outperforming others.

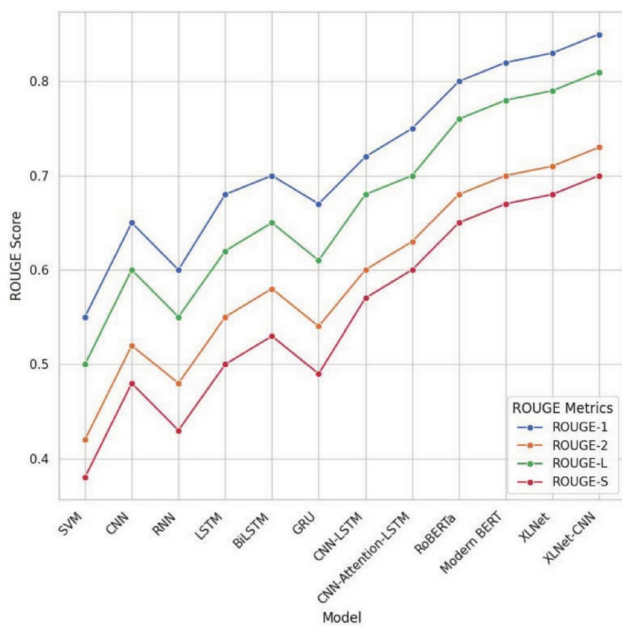
While mean performance provides insight, statistical testing is necessary to determine whether these differences are truly significant. Table 10 addresses this by reporting pairwise significance tests comparing the accuracy of each model to the XLNet-CNN. Both paired t tests and Wilcoxon signed-rank tests were conducted, offering complementary parametric and non-parametric perspectives. The results show that for nearly all models, the p-values are extremely small (often $< 1e-09$ for the t-test and consistently < 0.01 for the Wilcoxon test). This indicates that the improvements of XLNet-CNN over other models are not only numerically superior but also statistically significant. The only exception is when comparing XLNet-CNN to itself, where no values are reported, as expected. Importantly, even state-of-the-art models such as GPT, BERT, and XLNet show statistically significant differences when compared against XLNet-CNN, with p-values as low as $3.6e-04$ and $1.7e-05$. These findings reinforce the robustness of the proposed hybrid architecture, confirming that the observed gains are unlikely to be due to random variation.

To provide a global perspective, Table 11 presents the results of ANOVA (Analysis of Variance) across all models for each evaluation metric. ANOVA tests whether there are statistically significant differences in the means of multiple groups. The results are striking: all metrics show extremely high F-scores and vanishingly small p-values (well below 0.001). For example, Precision has an F-score of 608.38 ($p = 6.74e-107$), Accuracy has an F-score of 325.64 ($p = 3.37e-90$), and BLEU reaches a staggering F-score of 236,948 ($p = 1.25e-269$). These results confirm that differences across models are not only present but also highly statistically significant across all dimensions of evaluation. The extremely low p-values further validate the claim that model architecture has a profound effect on performance in AI-generated text detection.

In summary, the results from Tables 9, 10 and 11 provide strong empirical and statistical evidence of the superiority of the proposed XLNet-CNN model. Table 9 demonstrates its numerical dominance across all key metrics, Table 10

Table 12 Comparison of ROUGE scores for different models in detecting AI-generated essays

Model	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-S
SVM	0.55	0.42	0.50	0.38
CNN	0.65	0.52	0.60	0.48
RNN	0.60	0.48	0.55	0.43
LSTM	0.68	0.55	0.62	0.50
BiLSTM	0.70	0.58	0.65	0.53
GRU	0.67	0.54	0.61	0.49
CNN-LSTM	0.72	0.60	0.68	0.57
CNN-Attention-LSTM	0.75	0.63	0.70	0.60
RoBERTa	0.80	0.68	0.76	0.65
Modern BERT	0.82	0.70	0.78	0.67
XLNet	0.83	0.71	0.79	0.68
XLNet-CNN	0.85	0.73	0.81	0.70

**Fig. 11** Comparison of ROUGE scores across models

confirms that these improvements are statistically significant compared to other baselines, and Table 11 shows that differences across models are globally significant. The findings highlight three important trends: (1) traditional models like SVM and basic deep models lag significantly behind in this task, (2) hybrid approaches such as CNN-Attention-LSTM show noticeable improvements due to their ability to model sequential and hierarchical text features, and (3) Transformer-based architectures, particularly XLNet-CNN, achieve state-of-the-art performance by combining contextual richness with local feature extraction. These results underscore the importance of hybrid deep learning strategies in tackling the challenges of AI-generated text detection and contribute evidence that architectural innovations yield meaningful and statistically reliable improvements.

4.4.4 Visualization hybrid models comparison of ROUGE scores

Visualization of Hybrid Models' Comparison of ROUGE Scores provides a graphical representation of how different hybrid models perform in terms of their ROUGE scores. In the context of detecting AI-generated versus human-generated essays, hybrid models (e.g., combining CNN with LSTM or Transformer models) are compared to assess how well they capture linguistic features and overall text quality. The visualization offers insights into which hybrid models achieve higher ROUGE scores across different metrics (ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-S), which are used to measure the quality and relevance of text generation in question-answering systems. By comparing these scores, one can evaluate the effectiveness of hybrid models for this specific task.

Table 12 and Fig. 11 compare the ROUGE scores (ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-S) for various models used to detect AI-generated versus human-generated essays. The comparison of different models based on ROUGE scores reveals the varying strengths and weaknesses of traditional machine learning models versus modern deep learning architectures. The traditional SVM model performs relatively modestly, with an ROUGE-1 score of 0.55 and an ROUGE-2 score of 0.42, which are considerably lower than those of more advanced models. SVM's performance is highly dependent on feature extraction methods, and it struggles to capture complex patterns in sequential data. On the other hand, CNNs, which focus on local patterns, outperform SVM by a significant margin, achieving a ROUGE-1 score of 0.65 and a ROUGE-2 score of 0.52. This indicates that CNNs are better at identifying useful features in text, particularly for shorter sequences. Recurrent Neural Networks (RNNs), with a ROUGE-1 score of 0.60 and ROUGE-2 score of 0.48, show an improvement over SVM and CNNs, particularly in handling sequential data. However, they still struggle with long-term dependencies, limiting their performance in more complex tasks.

Long Short-Term Memory (LSTM) networks significantly improve upon RNNs, handling long-term dependencies better, and score higher with ROUGE-1 of 0.68 and ROUGE-2 of 0.55.

Bidirectional LSTMs (BiLSTM) further enhance accuracy by capturing both forward and backward context, achieving the highest performance among traditional deep learning architectures, with a ROUGE-1 score of 0.70 and a ROUGE-2 score of 0.58. Gated Recurrent Units (GRU), while less powerful than LSTMs, offer a simplified version with competitive performance (ROUGE-1 of 0.67 and ROUGE-2 of 0.54). The combination of CNN and LSTM (CNN-LSTM) increases performance even further, with ROUGE-1 reaching 0.72. Attention mechanisms incorporated into CNN-LSTM models (CNN-Attention-LSTM) boost performance, particularly in ROUGE-L, achieving a score of 0.70. The pretrained models, such as RoBERTa (ROUGE-1 of 0.80) and modern BERT (ROUGE-1 of 0.82), excel in contextual understanding, with XLNet (ROUGE-1 of 0.83) showing even better results. The XLNet-CNN hybrid combines the best of both worlds, with the highest ROUGE scores of all models, particularly excelling in both contextual accuracy and pattern extraction, indicating its superior performance in complex text generation and understanding tasks.

5 Challenges and limitations

Despite the strong performance of the proposed XLNet-CNN hybrid model, several challenges and limitations remain:

1. **Evolving nature of AI-generated text:** With the rapid advancement of large language models (LLMs) such as GPT-4, AI-generated text increasingly resembles human writing. This makes it difficult for detection systems like XLNet-CNN to consistently identify subtle linguistic artifacts, requiring continuous retraining and adaptation.
2. **Generalization and adaptability:** although the XLNet-CNN model demonstrates robust results on the EnglishQA Essays Corpus, its ability to generalize across different domains, academic subjects, and multilingual contexts is limited. The reliance on domain-specific datasets may restrict its effectiveness when applied to unseen or diverse text sources.
3. **Need for robust training strategies:** high accuracy on benchmark datasets does not eliminate the risk of overfitting. Future work must incorporate adversarial training, dynamic learning strategies, and real-time

monitoring to improve adaptability and resilience against increasingly sophisticated AI-generated essays.

6 Conclusion

This paper presented a fine-tuned XLNet-CNN hybrid framework for detecting AI-generated essays, addressing the pressing challenge of maintaining academic integrity in the era of large language models (LLMs). By combining XLNet's generalized autoregressive pretraining and bidirectional context learning with the convolutional neural network's strength in local feature extraction, the proposed approach consistently outperformed traditional classifiers and transformer-based baselines on the EnglishQA Essays Corpus. Experimental results demonstrated strong detection capability, with the model achieving accuracy of 0.98, recall of 0.96, F2-score of 0.97, and precision of 1.00, while eliminating false negatives. These findings underscore the effectiveness of hybrid architectures in capturing both semantic coherence and stylistic artifacts that distinguish human-written from AI-generated text. Comparative evaluations against widely used detection tools further confirmed the adaptability and reliability of the framework across diverse writing styles and contexts. Overall, the XLNet-CNN model offers a scalable and practical solution for safeguarding educational integrity in high-stakes academic settings.

7 Future research directions

Building upon the identified challenges, several directions for future research can be explored:

1. **Cross-domain and multilingual expansion:** extending the XLNet-CNN model to handle multilingual datasets and domain-specific academic writing (e.g., scientific, technical, and humanities essays) will enhance its generalization ability and make it more applicable in global educational contexts.
2. **Integration of hybrid architectures:** combining XLNet-CNN with emerging architectures such as state space models (SSMs) or attention-free transformers can further improve both efficiency and detection accuracy. Exploring hybrid ensembles may also help balance robustness and inference speed.
3. **Adversarial and real-time training:** future research should incorporate adversarial training and real-time monitoring frameworks to adapt to evolving LLMs. This will enable the detection system to remain resilient against novel generative models that continuously refine their outputs.

4. Explainability and educational tools: enhancing the interpretability of model predictions will provide educators with actionable insights, helping them understand why a text is flagged as AI-generated. Integrating explainable AI (xAI) techniques could increase trust and adoption in academic environments.
5. Ethical and policy considerations: beyond technical development, future work should also address ethical concerns and collaborate with institutions to formulate guidelines that balance the responsible use of AI tools in education while safeguarding academic integrity.

Author contributions MP conceptualized and designed the study, performed the analysis, and wrote the original draft. PPV contributed to data collection and interpretation and critically reviewed the manuscript. SKB AND PPV supervised the project and was involved in the final editing of the manuscript. All authors read and approved the final manuscript.

Data availability The data that support the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Saul D (2023) ChatGPT parent Open AI Gets ‘game changing’ multibillion-dollar boost from Microsoft. *Forbes*, Jersey
2. Fyfe P (2023) How to cheat on your final paper: assigning ai for student writing. *AI & Soc* 38(4):1395–1405
3. Yeadon W, Inyang O-O, Mizouri A, Peach A, Testrow CP (2023) The death of the short-form physics essay in the coming ai revolution. *Phys Educ* 58(3):035027
4. Sharples M (2022) Automated essay writing: an AIED opinion. *Int J Artif Intell Educ* 32(4):1119–1126
5. Waisberg E, Ong J, Masalkhi M, Kamran SA, Zaman N, Sarker P, Lee AG, Tavakkoli A (2023) Gpt-4: a new era of artificial intelligence in medicine. *Ir J Med Sci* (1971) 192(6):3197–3200
6. Floridi L, Chiriatti M (2020) Gpt-3: its nature, scope, limits, and consequences. *Mind Mach* 30:681–694
7. Yadagiri A, Pakray P (2025) Deep learning strategies for identifying machine-generated text. 2025 19th International conference on ubiquitous information management and communication (IMCOM). IEEE, Geneva, pp 1–8
8. Lin C-Y, Chien T-W, Chen Y-H, Lee Y-L, Su S-B (2022) An app to classify a 5-year survival in patients with breast cancer using the convolutional neural networks (CNN) in Microsoft excel: development and usability study. *Medicine* 101(4):28697
9. Sherstinsky A (2020) Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. *Physica D* 404:132306
10. Song X, Liu Y, Xue L, Wang J, Zhang J, Wang J, Jiang L, Cheng Z (2020) Time-series well performance prediction based on long short-term memory (LSTM) neural network model. *J Petrol Sci Eng* 186:106682
11. Liu G, Guo J (2019) Bidirectional LSTM with attention mechanism and convolutional layer for text classification. *Neurocomputing* 337:325–338
12. Rana R (2016) Gated recurrent unit (gru) for emotion classification from noisy speech. arXiv preprint [arXiv:1612.07778](https://arxiv.org/abs/1612.07778)
13. Alhussein M, Aurangzeb K, Haider SI (2020) Hybrid CNN-LSTM model for short-term individual household load forecasting. *Ieee Access* 8:180544–180557
14. Saqware GJ (2024) Hybrid deep learning model integrating attention mechanism for the accurate prediction and forecasting of the cryptocurrency market. *Operations research forum*. Springer, Cham, p 19
15. Semary NA, Ahmed W, Amin K, Plawiak P, Hammad M (2023) Improving sentiment classification using a Roberta-based hybrid model. *Front Hum Neurosci* 17:1292010
16. Yang Z (2019) Xlnet: generalized autoregressive pretraining for language understanding. arXiv preprint [arXiv:1906.08237](https://arxiv.org/abs/1906.08237)
17. Gehrmann S, Strobelt H, Rush AM (2019) Gltr: statistical detection and visualization of generated text. arXiv preprint [arXiv:1906.04043](https://arxiv.org/abs/1906.04043)
18. Sadasivan VS, Kumar A, Balasubramanian S, Wang W, Feizi S (2023) Can ai-generated text be reliably detected? arXiv preprint [arXiv:2303.11156](https://arxiv.org/abs/2303.11156)
19. Goldberg Y, Hirst G (2017) Neural network methods in natural language processing. Morgan and Claypool Publishers, San Rafael, p 69
20. Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. arXiv preprint [arXiv:1301.3781](https://arxiv.org/abs/1301.3781)
21. Ling W, Dyer C, Black AW, Trancoso I (2015) Two/too simple adaptations of word2vec for syntax problems. In: *Proceedings of the 2015 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp 1299–1304
22. Neumann M, Iyyer M, Gardner M, Clark C, Lee K, Zettlemoyer L (2018) Deep contextualized word representations. arXiv preprint [arXiv:1802.05365](https://arxiv.org/abs/1802.05365)
23. Howard J, Ruder S (2018) Universal language model fine-tuning for text classification. arXiv preprint [arXiv:1801.06146](https://arxiv.org/abs/1801.06146)
24. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. *Adv Neural Inf Process Syst* 30
25. Devlin J, Chang M-W, Lee K, Toutanova K (2018) Bert: pre-training of deep bidirectional transformers for language understanding. arXiv preprint [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)
26. Radford A, Narasimhan K, Salimans T, Sutskever I (2018) Improving language understanding with unsupervised learning. 2018. URL: <https://openai.com/research/language-unsupervised>
27. Lewis M, Liu Y, Goyal N, Ghazvininejad M, Mohamed A, Levy O, Stoyanov V, Zettlemoyer L (2019) Bart: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. arXiv preprint [arXiv:1910.13461](https://arxiv.org/abs/1910.13461)
28. Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, Levy O, Lewis M, Zettlemoyer L, Stoyanov V (2019) Roberta: a robustly optimized bert pretraining approach. arXiv preprint [arXiv:1907.11692](https://arxiv.org/abs/1907.11692)
29. Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. *J Mach Learn Res* 21(140):1–67
30. Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2023) Large language models for software engineering: a systematic literature review. arXiv preprint [arXiv:2308.10620](https://arxiv.org/abs/2308.10620)
31. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020)

- Language models are few-shot learners. *Adv Neural Inf Process Syst* 33:1877–1901
32. Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, et al (2023) Gpt-4 technical report. arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774)
 33. Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M-A, Laroix T, Rozière B, Goyal N, Hambro E, Azhar F, et al (2023) Llama: open and efficient foundation language models. arXiv preprint [arXiv:2302.13971](https://arxiv.org/abs/2302.13971)
 34. Reynolds L, McDonnell K (2021) Prompt programming for large language models: beyond the few-shot paradigm. In: *Extended abstracts of the 2021 CHI conference on human factors in computing systems*, pp 1–7
 35. Trummer I (2022) Codexdb: synthesizing code for query processing from natural language instructions using GPT-3 codex. *Proc VLDB Endow* 15(11):2921–2928
 36. White J, Fu Q, Hays S, Sandborn M, Olea C, Gilbert H, Elnashar A, Spencer-Smith J, Schmidt DC (2023) A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint [arXiv:2302.11382](https://arxiv.org/abs/2302.11382)
 37. Cascella M, Montomoli J, Bellini V, Bignami E (2023) Evaluating the feasibility of Chatgpt in healthcare: an analysis of multiple clinical and research scenarios. *J Med Syst* 47(1):33
 38. Levin G, Meyer R, Kadoch E, Brezinov Y (2023) Identifying chatgpt-written OBGYN abstracts using a simple tool. *Am J Obstet Gynecol MFM* 5(6):100936
 39. Gupta R, Pande P, Herzog I, Weisberger J, Chao J, Chaiyasate K, Lee ES (2023) Application of chatgpt in cosmetic plastic surgery: ally or antagonist? *Aesthetic Surg J* 43(7):587–590
 40. Lahat A, Shachar E, Avidan B, Shatz Z, Glicksberg BS, Klang E (2023) Evaluating the use of large language model in identifying top research questions in gastroenterology. *Sci Rep* 13(1):4164
 41. Lyu Q, Tan J, Zapadka ME, Ponnatapura J, Niu C, Myers KJ, Wang G, Whitlow CT (2023) Translating radiology reports into plain language using chatgpt and gpt-4 with prompt learning: promising results, limitations, and potential. arXiv preprint [arXiv:2303.09038](https://arxiv.org/abs/2303.09038)
 42. Thorp HH (2023) ChatGPT is fun, but not an author. American Association for the Advancement of Science, Washington
 43. Van Dis EA, Bollen J, Zuidema W, Van Rooij R, Bockting CL (2023) Chatgpt: five priorities for research. *Nature* 614(7947):224–226
 44. Germain M, Gregor K, Murray I, Larochelle H (2015) Made: masked autoencoder for distribution estimation. In: *International Conference on Machine Learning*, PMLR, pp 881–889.
 45. Uria B, Côté M-A, Gregor K, Murray I, Larochelle H (2016) Neural autoregressive distribution estimation. *J Mach Learn Res* 17(205):1–37
 46. Fedus W, Goodfellow I, Dai AM (2018) Maskgan: better text generation via filling in the . arXiv preprint [arXiv:1801.07736](https://arxiv.org/abs/1801.07736)
 47. Ouyang X, Zhou P, Li CH, Liu L (2015) Sentiment analysis using convolutional neural network. In: *2015 IEEE international conference on computer and information technology; ubiquitous computing and communications; dependable, autonomic and secure computing; pervasive intelligence and computing*, pp. 2359–2364. IEEE
 48. Pennington J, Socher R, Manning CD (2014) Glove: global vectors for word representation. In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pp. 1532–1543
 49. Li W (2020) Analysis of semantic comprehension algorithms of natural language based on robot's questions and answers. In: *2020 IEEE International conference on advances in electrical engineering and computer applications (AEECA)*, IEEE pp. 1021–1024.
 50. Deguang C, Jinlin M, Ziping M, Jie Z (2021) Review of pre-training techniques for natural language processing. *J Front Comput Sci Tech* 15(8):1359
 51. Essays Corpus datasets (2023) <https://www.kaggle.com/competitions/llm-detect-ai-generated-text/data>. Accessed 31 Oct 2023
 52. EnglishQA Essays Corpus datasets (2023) <https://www.kaggle.com/datasets/thedrcat/daigt-proper-train-dataset>. Accessed 11 May 2023
 53. Crossley SA, Tian Y, Baffour P, Franklin A, Benner M, Boser U (2024) A large-scale corpus for assessing written argumentation: persuade 2.0. *Assess Writ* 61:100865
 54. EnglishQA Essays Corpus datasets. <https://www.kaggle.com/datasets/nbroad/daigt-data-llama-70b-and-falcon180b>. Accessed 11 May 2024
 55. Abbas HM (2025) A novel approach to automated detection of ai-generated text. *J Al-Qadisiyah Comput Sci Math* 17(1):1–17
 56. EnglishQA Essays Corpus datasets (2024) <https://www.kaggle.com/datasets/darraghdog/hello-claude-1000-essays-from-anthropi>. Accessed 11 May 2024
 57. EnglishQA Essays Corpus datasets (2023) (<https://www.kaggle.com/datasets/alejopaullier/daigt-external-dataset>). Accessed 11 May 2023
 58. EnglishQA Essays Corpus datasets. <https://www.kaggle.com/datasets/kingki19/llm-generated-essay-using-palm-from-google-gen-ai>. Accessed 11 May 2024
 59. EnglishQA Essays Corpus datasets. <https://www.kaggle.com/datasets/radek1/llm-generated-essays>. Accessed 11 May 2024
 60. Naseem S, Mahmood T, Asif M, Rashid J, Umair M, Shah M (2021) Survey on sentiment analysis of user reviews. In: *2021 International Conference on Innovative Computing (ICIC)*, IEEE pp 1–6.
 61. Chen A, Stanovsky G, Singh S, Gardner M (2019) Evaluating question answering evaluation. In: *Proceedings of the 2nd Workshop on Machine Reading for Question Answering*, pp 119–124
 62. Jelinek F (1998) *Statistical methods for speech recognition*. MIT press, Cambridge
 63. Papineni K, Roukos S, Ward T, Zhu W-J (2002) Bleu: a method for automatic evaluation of machine translation. In: *Proceedings of the 40th mof the association for computational linguistics*, pp 311–318
 64. Sokolova M, Lapalme G (2009) A systematic analysis of performance measures for classification tasks. *Inf Process Manag* 45(4):427–437

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.